

Artificial Neural Networks

CSA4002

Course Objectives:

- To understand the biological neural network and to model equivalent neuron models.
- To understand the architecture, learning algorithm and issues of various feed forward and feedback neural networks.
- It deals with Associate Memories and introduces single and multi-layer perception networks

Course Outcomes

- Model Neuron and Neural Network, and to analyze ANN learning, and its applications.
- Perform learning process models.
- Develop different single layer Perception learning algorithms
- Develop different multiple layer Perception learning algorithms
- Develop associative memory model

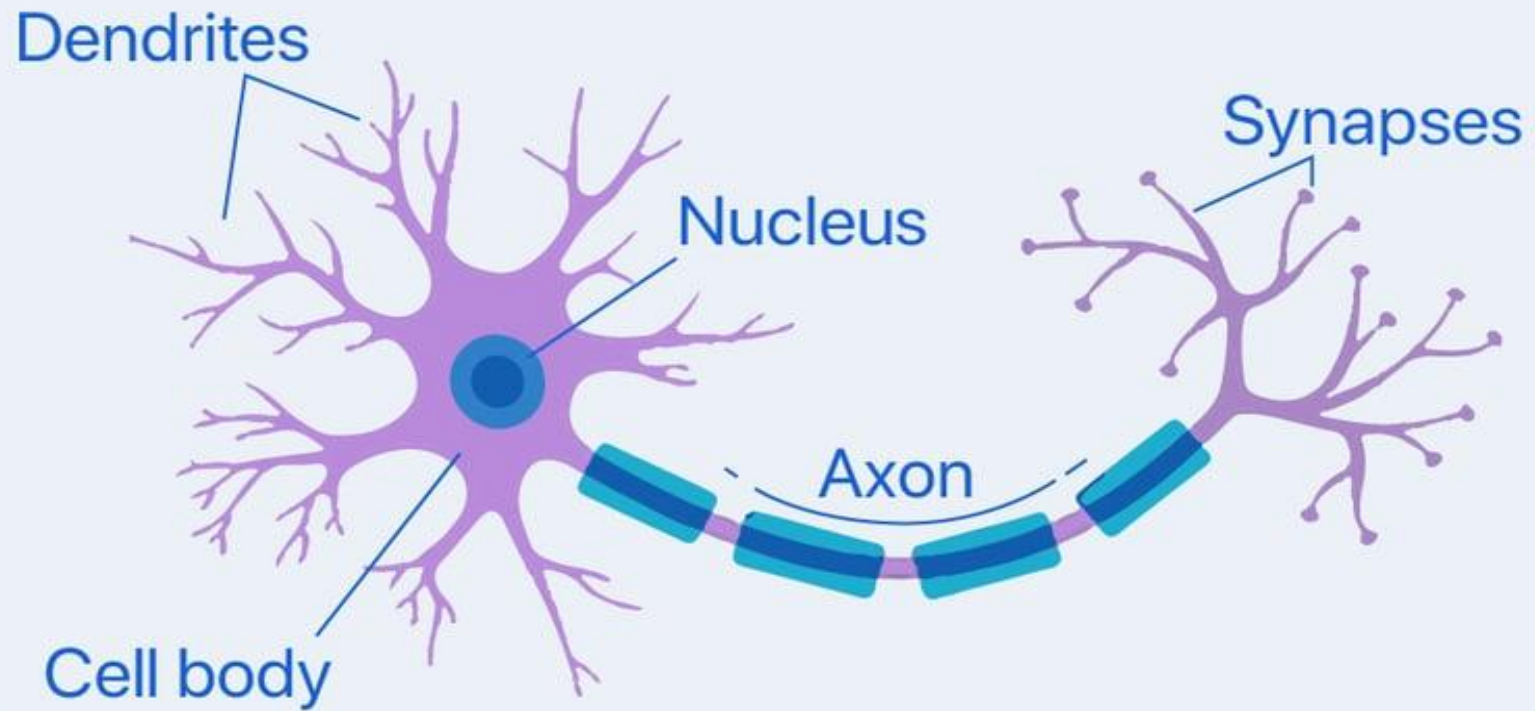
Introduction

- Introduction to Neural Networks:
- Neural Network, Human Brain, Models of Neuron, Neural networks viewed as directed graphs, Biological Neural Network, Artificial neuron, Artificial Neural Network architecture, ANN learning, analysis, and ,applications, Historical notes

What is a Neural Network?

- A **Neural Network** is a type of **machine learning model** inspired by the structure and function of the **human brain**.
- It consists of **artificial neurons** (also called nodes or units) that process data and learn patterns to perform tasks like classification, prediction, and recognition.

Biological Neuron



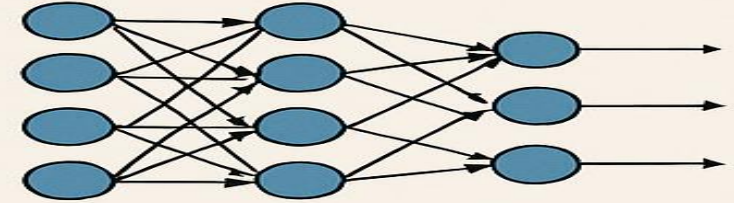
Artificial Neural Networks

- Artificial Neural Networks (ANNs) are computer systems designed to mimic how the human brain processes information.
- Just like the brain uses neurons to process data and make decisions, ANNs use artificial neurons to analyze data, identify patterns, and make predictions.
- These networks consist of layers of interconnected neurons that work together to solve complex problems.
- The key idea is that ANNs can "learn" from the data they process, just as our brain learns from experience.
- They are used in various applications from recognizing images to making personalized recommendations.

HUMAN BRAIN vs NEURAL NETWORK (ANN)



HUMAN BRAIN

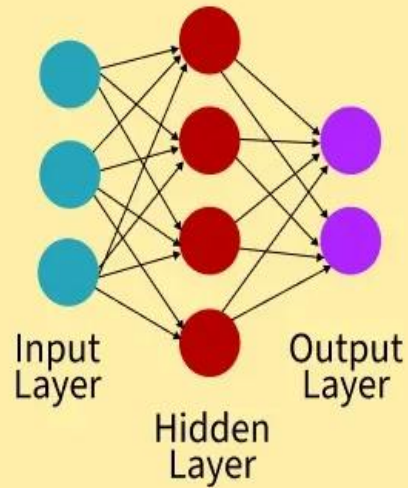


NEURAL NETWORK (ANN)

Basic Unit	Neuron	Artificial neuron (Perceptron)
Structure	Dense network of ~86 billion neurons connected by synapses	Layers of nodes (input, hidden, output) connected by weights
Signal Transmission	Electrochemical signals via synapses	Numerical values passed via weighted connections
Learning Mechanism	Synaptic plasticity (changing connection strength through experience)	Weight adjustment using algorithms (e.g., backpropagation)
Learning Style	Unsupervised and experience-driven (e.g., emotions, memory)	Typically supervised, requires labeled data
Processing Type	Massively parallel and energy-efficient	Simulated parallelism, high power consumption (GPU/TPU needed)
Adaptability	Tolerant to noise, graceful degradation	Prone to overfitting, sensitive to input variation
Error Handling	Evolved through biological processes over millions of years	Engineered by humans, based on biological inspiration
Consciousness & Emotions	Has self-awareness, emotions, creativity	No self-awareness or emotion: follows programmed logic
Energy Consumption	~20W for entire brain	Hundreds of watts for large models

What Are Artificial Neural Networks?

- Simulate the human brain's information processing.
- Consist of input, hidden and output layers.
- Learn patterns and make predictions from data.



Feedforward Neural Networks (FNN)



Convolutional Neural Networks (CNN)

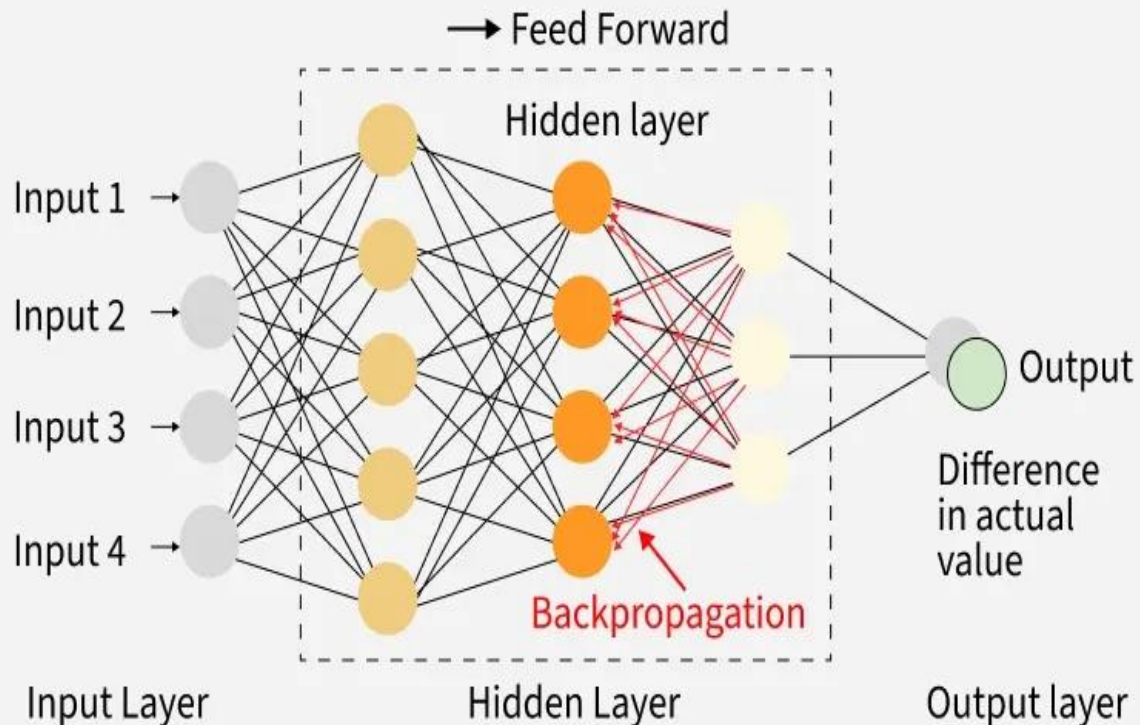
Types of Artificial Neural Networks



Recurrent Neural Networks (RNN)



Modular Neural Networks (MNN)



01



Social Media

02



Healthcare

03



Finance

Applications of Artificial Neural Networks



Personal Assistants



Marketing



Retail

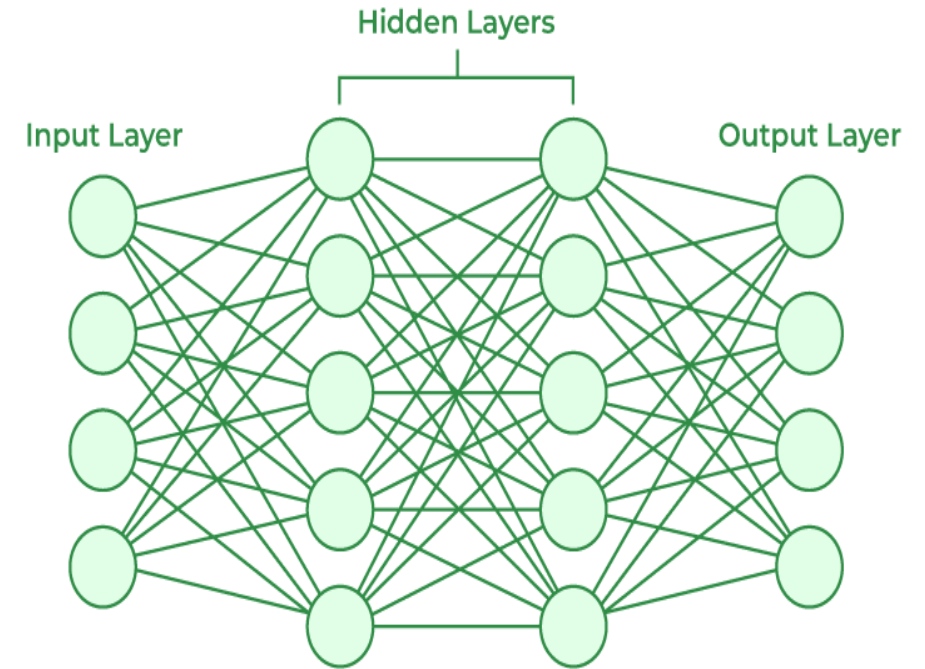
04

05

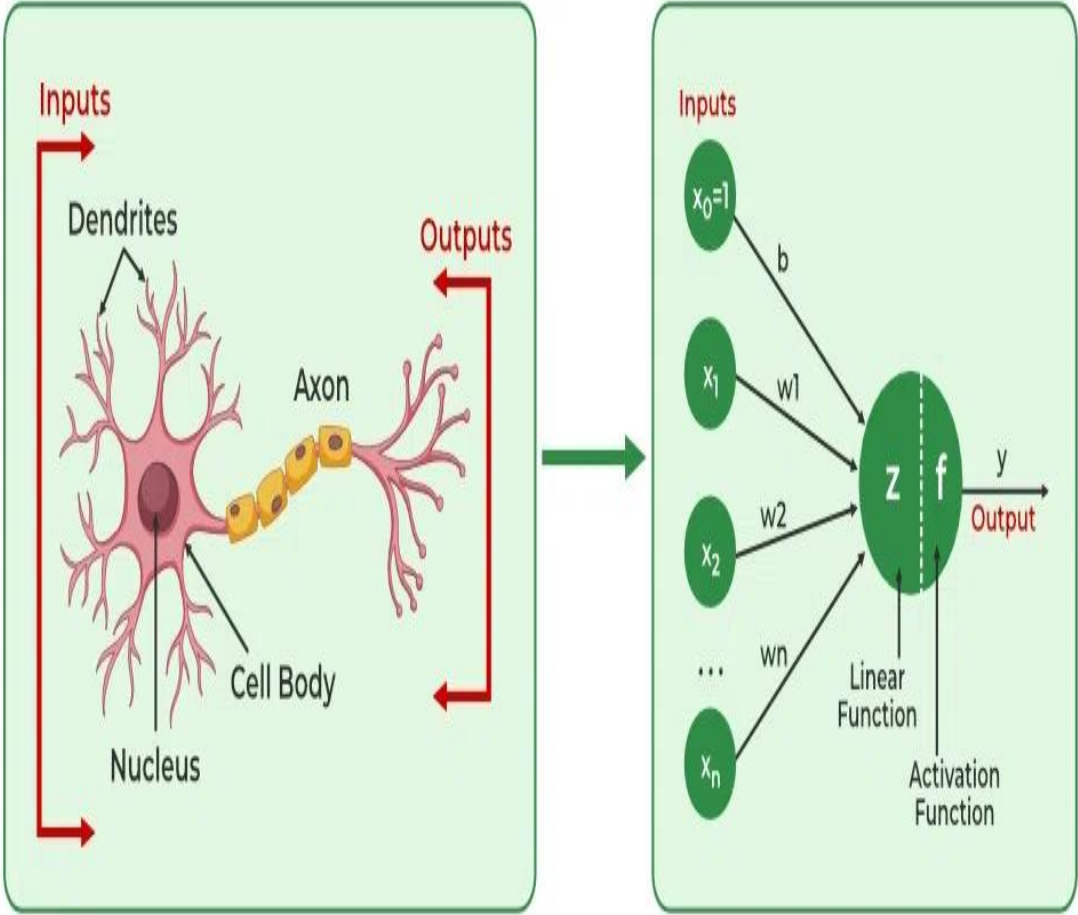
06

Basic Structure of a Neural Network

- **Input Layer**
 - Receives raw data (e.g., images, text, numbers)
 - Each neuron corresponds to one feature
- **Hidden Layers**
 - Perform computations using **weights** and **activation functions**
 - Extract patterns and relationships in the data
 - More hidden layers = deeper network (**Deep Learning**)
- **Output Layer**
 - Produces final result (e.g., category, value)
 - The number of neurons depends on the task (e.g., binary output = 1 neuron)



Aspect	Biological Neurons	Artificial Neurons
Structure	Dendrites: Receive signals from other neurons.	Input Nodes: Receive data and pass it on to the next layer.
	Cell Body (Soma): Processes the signals.	Hidden Layer Nodes: Process and transform the data.
	Axon: Transmits processed signals to other neurons.	Output Nodes: Produce the final result after processing.
Connections	Synapses: Links between neurons that transmit signals.	Weights: Connections between neurons that control the influence of one neuron on another.
Learning Mechanism	Synaptic Plasticity: Changes in synaptic strength based on activity over time.	Backpropagation: Adjusts the weights based on errors in predictions to improve future performance.
Activation	Activation: Neurons fire when signals are strong enough to reach a threshold.	Activation Function: Maps input to output, deciding if the neuron should fire based on the processed data.



Term	Meaning
Neuron	Small processing unit that takes inputs and produces an output
Weight	Determines the strength of connection between neurons
Bias	Extra value added to the neuron output to help with learning
Activation Function	Introduces non-linearity to model complex patterns
Epoch	One full pass through the entire dataset during training

Training and Testing:

- During training, the network is shown examples like images of cats and learns to recognize patterns in them.
- After training, the network is tested on new data to check its performance. The better the network is trained, the more accurately it will predict new data.

Artificial Neural Networks (ANNs)

- Artificial Neural Networks are computational models inspired by the human brain's neural architecture.
- The simplest form of ANN follows a feed-forward mechanism where data flows from input to output without looping back.
- These networks consist of interconnected layers: input layers that receive data, hidden layers that process it and output layers that produce the final result.

Advantages of ANNs:

- **Versatile Learning:** ANNs can handle both linear and non-linear data which makes them applicable across diverse domains.
- **Forecasting:** They are sensitive to complex patterns making them effective in time series forecasting, such as predicting stock prices or economic trends.

Disadvantages of ANNs

- **Lack of Interpretability:** Due to their black-box nature, it is difficult to understand how decisions are made within the network.
- **Hardware Dependence:** ANNs require heavy computational resources which can limit their scalability in certain environments.

Biological Neural Networks (BNNs)

- Biological Neural Networks are the foundation of cognition in living organisms. A biological neuron comprises dendrites, a cell body, and an axon. Dendrites receive signals from other neurons, the soma integrates these inputs, and the axon transmits the resulting signal to subsequent neurons via synapses.
- **Advantages of BNNs:**
- **Input Handling:** Biological synapses are capable of interpreting and integrating a wide variety of stimuli/inputs.
- **Parallel Processing:** BNNs are efficient at processing massive amounts of information simultaneously, enabling rapid responses.

Disadvantages of BNNs:

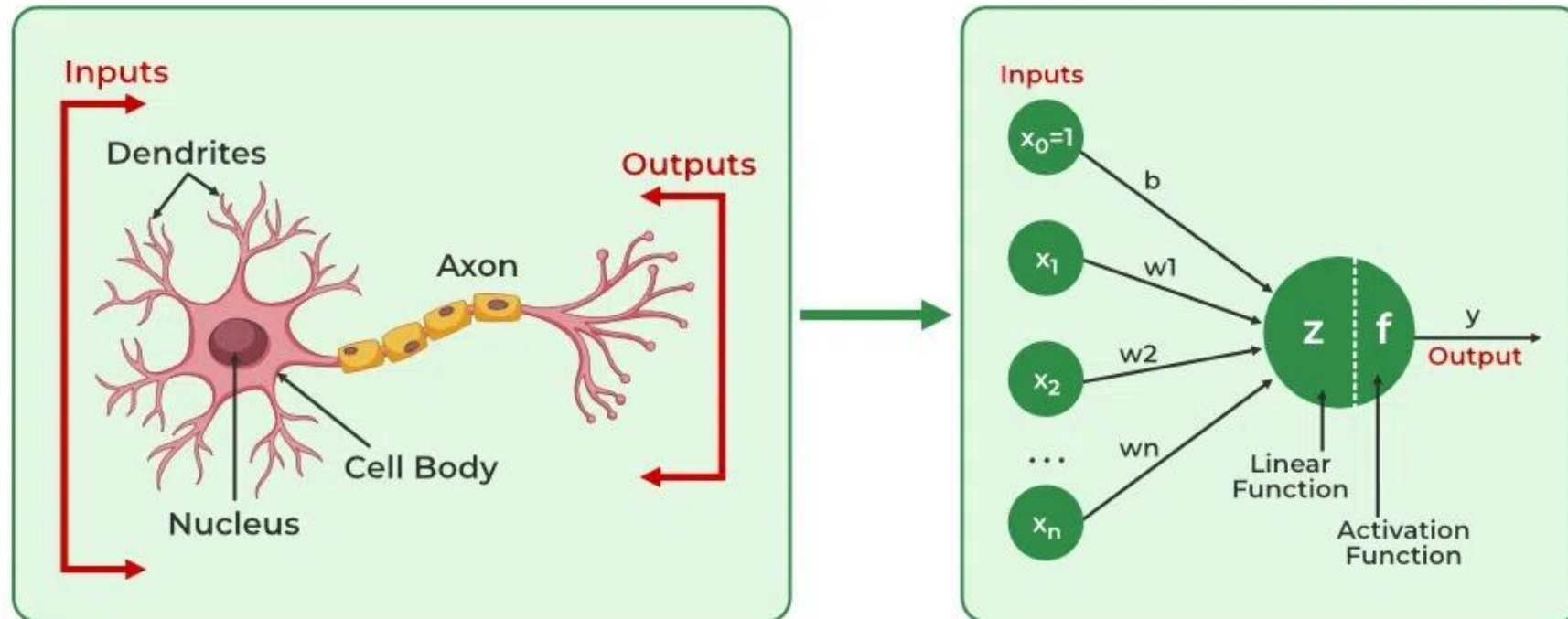
- **Lack of Central Control:** Unlike artificial systems, BNNs lack a clear central processing unit, which can make control mechanisms less structured.
- **Slower Processing:** BNNs operate at slower speeds compared to silicon-based systems due to the nature of electrochemical transmission.

Comparison Table Between ANN and BNN

Parameter	Artificial Neural Network (ANN)	Biological Neural Network (BNN)
Structure	Input -> Hidden Layers -> Output	Dendrites -> Cell Body -> Axon
Learning	Requires structured, formatted data	Can handle ambiguous, noisy inputs
Computing	Centralized, sequential, program-driven	Distributed, parallel, self-learning
Reliability	Vulnerable to damage or failure	High fault tolerance and robustness
Operating Context	Well-defined, constrained environments	Unconstrained, often unpredictable

Feature	Artificial Neural Networks (ANNs)	Biological Neural Networks (BNNs)
Basic Unit	Artificial neuron (perceptron)	Biological neuron
Structure	Layered architecture (input, hidden, output)	Highly complex, non-layered, massive interconnections
Learning Mechanism	Backpropagation, gradient descent, supervised learning	Synaptic plasticity, neurogenesis, unsupervised and reinforcement learning
Speed	Fast in computation but slower learning (needs multiple epochs)	Slower computation but faster learning from fewer examples
Energy Efficiency	High power consumption (needs GPUs/TPUs)	Extremely energy-efficient (brain uses ~20W)
Fault Tolerance	Prone to errors, needs retraining	Highly tolerant to noise and damage
Flexibility	Task-specific, needs retraining for new tasks	Highly generalizable, learns and adapts to many tasks
Memory	Encoded in weights and biases	Distributed across neurons and synapses
Scalability	Limited by hardware and data	Scales naturally over development and experience
Development	Man-made, modeled using math and programming	Result of millions of years of evolution
Consciousness & Emotions	No consciousness, logic-based	Capable of emotions, awareness, creativity

Differences Between ANNs and BNNs



Difference between ANN and BNN

- Both natural intelligence and artificial intelligence works on networks of neurons.
- While Artificial Neural Networks (ANNs) draw ideas from Biological Neural Networks (BNNs) they still differ in structure, function and adaptability.

Neurons

- BNNs:** Composed of biological structures like dendrites and axons, with complex behavior and signal processing abilities.
- ANNs:** Use simplified models of neurons with a single output, focusing on numerical signal transformations through activation functions.

Learning

- BNNs:** Adapts based on learning, experience, and environmental factors.
- ANNs:** Use fixed mathematical weights that are adjusted during training but remain static during testing.

Neural Pathways

- BNNs:** Feature a highly complex web of adaptable pathways influenced by learning and memory.
- ANNs:** Have predefined pathways determined by network architecture and model design.

Aspect	Human Brain	Neural Network (ANN)
Basic Unit	Neuron	Artificial Neuron (Perceptron)
Structure	Dense network of ~86 billion neurons connected by synapses	Layers of nodes (input, hidden, output) connected by weights
Signal Transmission	Electrochemical signals via synapses	Numerical values passed via weighted connections
Learning Mechanism	Synaptic plasticity (changing connection strength through experience)	Weight adjustment using algorithms (e.g., backpropagation)
Learning Style	Unsupervised and experience-driven (e.g., emotions, memory)	Typically supervised, requires labeled data
Processing Type	Massively parallel and energy-efficient	Simulated parallelism, high power consumption (GPU/TPU needed)
Adaptability	Highly flexible, capable of generalizing from few examples	Needs large datasets and training cycles to generalize
Error Handling	Tolerant to noise, graceful degradation	Prone to overfitting, sensitive to input variation
Development	Evolved through biological processes over millions of years	Engineered by humans, based on biological inspiration
Consciousness & Emotions	Has self-awareness, emotions, creativity	No self-awareness or emotion; follows programmed logic
Energy Consumption	~20W for entire brain	Hundreds of watts for large models

Common Types of Neural Networks

- **Feedforward Neural Network (FNN):** Simple, used for basic tasks
- **Convolutional Neural Network (CNN):** Great for image recognition
- **Recurrent Neural Network (RNN):** Used for sequences (like text, time series)
- **Transformer Networks:** Advanced models for language and vision (e.g., GPT, BERT)

Applications

- Image & speech recognition
- Natural Language Processing (NLP)
- Recommendation systems
- Autonomous vehicles
- Medical diagnosis

Basic Structure of a Neural Network

Input Layer

Receives raw data
(e.g. images, text, numbers)
Each neuron
corresponds to one feature

Hidden Layers

Perform computations
using weights and
activation functions
Extract patterns and
relationships in the data

➔ How It Works (Simplified)

Forward Propagation

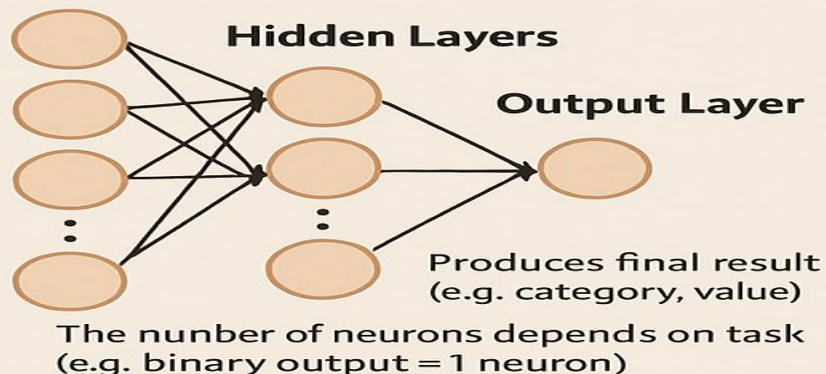
→ Data flows from input → hidden
layers → output

Each neuron computes a weighted
sum of inputs, applying a loss
function (like ReLU, Sigmoid)

Loss Calculation

→ Introduces non-linearity with
model complex patterns

A loss function measures
the error (e.g. Mean Squared
Error, Cross-Entropy)



🔑 Key Concepts

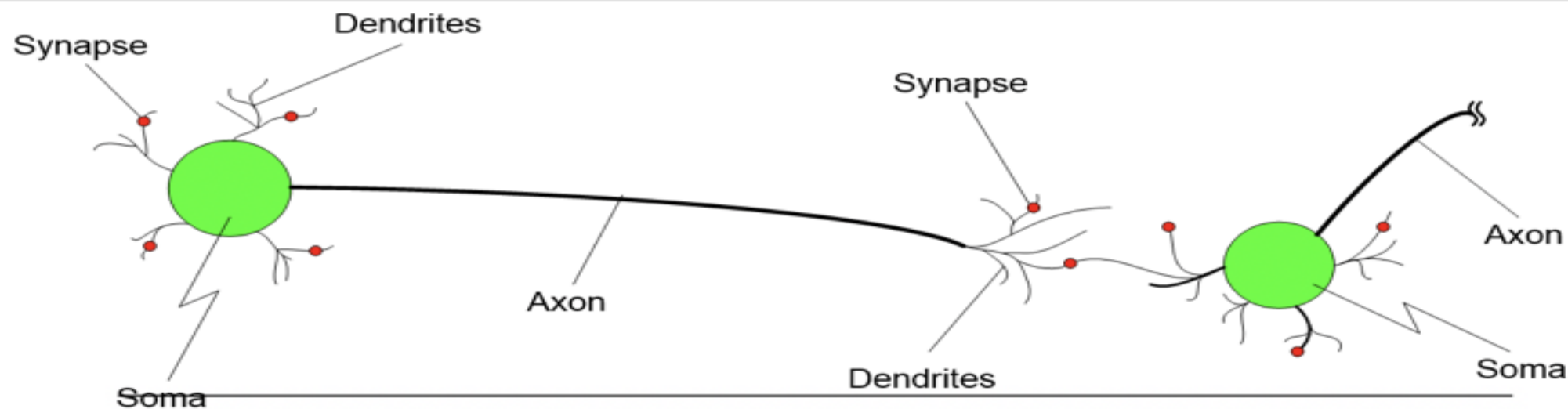
Term	Meaning
Neuron	Small processing unit that takes inputs and produces an output
Weight	Determines the strength of connection between neurons
Bias	Extra value added to the neuron output to help with learning
Activation Function	Introduces non-linearity to model complex patterns
Epoch	One full pass through the entire dataset during training

🧠 Common Types of Neural Networks

Term	Meaning
Neuron	Small processing unit that takes inputs and produces an output
Weight	Determines the strength of connection between neurons
Bias	Extra value added to the neuron output to help with learning

🧠 Applications

- image & speech recognition
- Natural Language Processing (NLP)
- Recommendation systems
- Autonomous vehicles



Biological

versus

Artificial

NNs

Soma

Node

Dendrites

Input

Axon

Output

Synapse

Weight

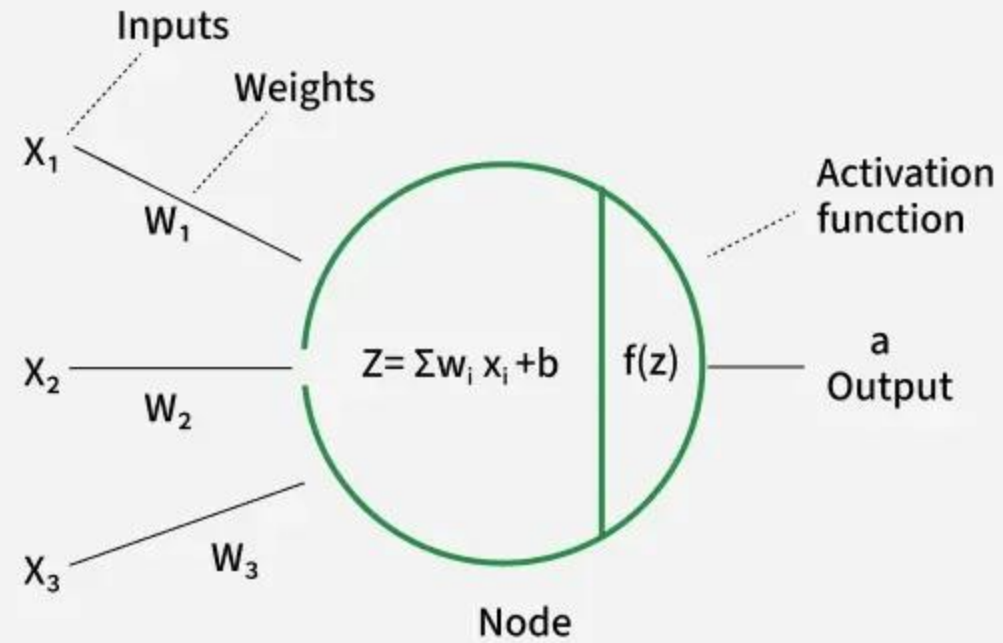
Slow

Fast

Many neurons (10^9)

Few neurons (~ 100 s)

Activation functions in Neural Networks



Introducing Non-Linearity in Neural Network

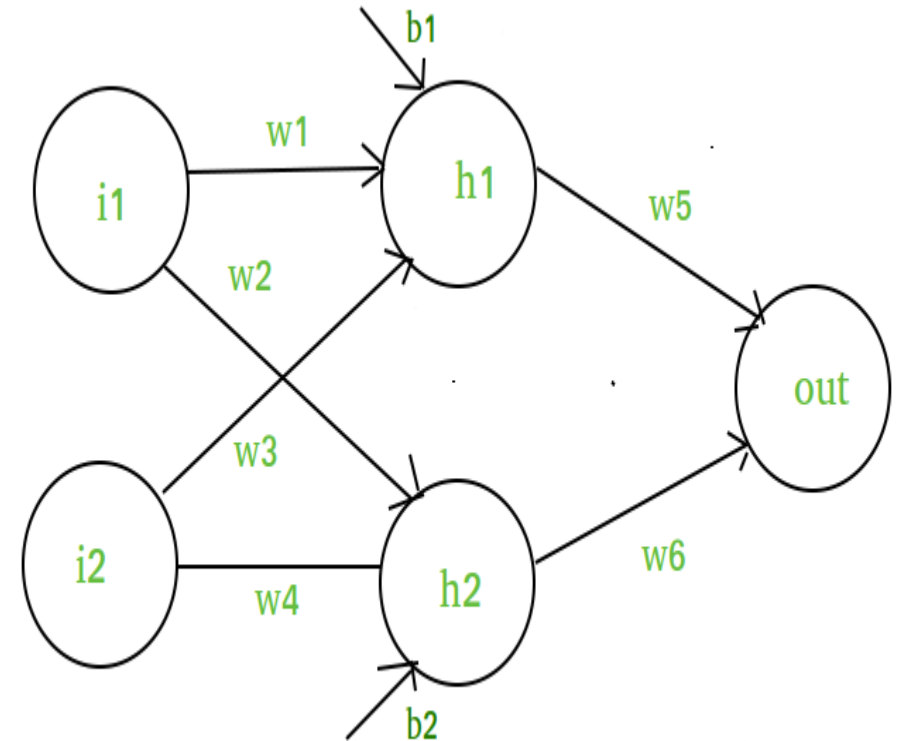
- Non-linearity means that the relationship between input and output is not a straight line. In simple terms the output does not change proportionally with the input. A common choice is the ReLU function defined as $\sigma(x) = \max(0, x)$.
- Imagine you want to classify apples and bananas based on their shape and color.
- If we use a linear function it can only separate them using a straight line.
- But real-world data is often more complex like overlapping colors, different lighting, etc.
- By adding a non-linear activation function like ReLU, Sigmoid or Tanh the network can create curved decision boundaries to separate them correctly.

Effect of Non-Linearity

- The inclusion of the ReLU activation function σ allows h_1 to introduce a non-linear decision boundary in the input space.
- This non-linearity enables the network to learn more complex patterns that are not possible with a purely linear model such as:
- Modeling functions that are not linearly separable.
- Increasing the capacity of the network to form multiple decision boundaries based on the combination of weights and biases.

Mathematical Proof of Need of Non-Linearity in Neural Networks

- To illustrate the need for non-linearity in neural networks with a specific example let's consider a network with two input nodes (i_1 and i_2), a single hidden layer containing neurons h_1 and h_2 and an output neuron (out).
- We will use w_1, w_2 as weights connecting the inputs to the hidden neuron and w_5 as the weight connecting the hidden neuron to the output. We'll also include biases (b_1 for the hidden neuron and b_2 for the output neuron) to complete the model.
- **Input Layer:** Two inputs i_1 and i_2 .
- **Hidden Layer:** Two neuron h_1 and h_2
- **Output Layer:** One output neuron.



The input to the hidden neuron h_1 is calculated as a weighted sum of the inputs plus a bias:

$$h_1 = i_1.w_1 + i_2.w_3 + b_1$$

$$h_2 = i_1.w_2 + i_2.w_4 + b_2$$

The output neuron is then a weighted sum of the hidden neuron's output plus a bias:

$$\text{output} = h_1.w_5 + h_2.w_6 + \text{bias}$$

Here, h_1 , h_2 and output are linear expressions.

In order to add non-linearity, we will be using sigmoid activation function in the output layer:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

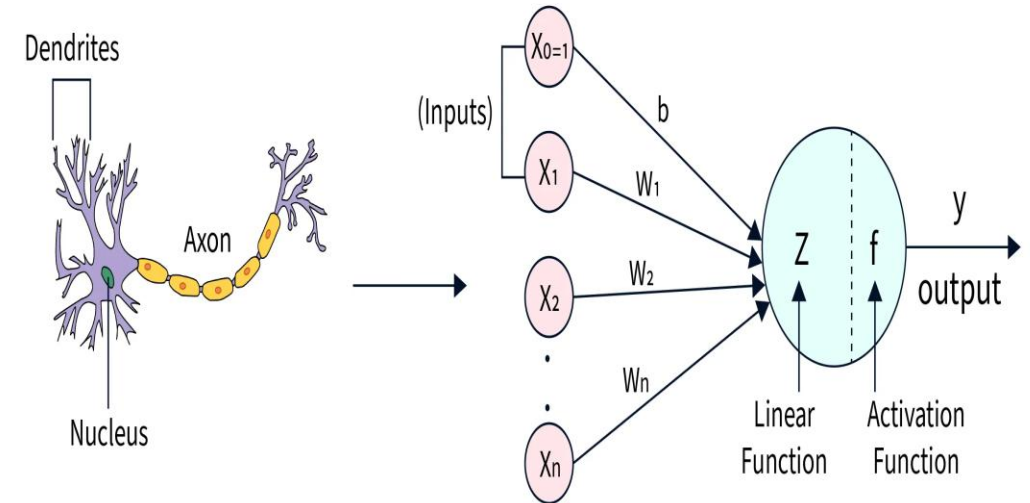
$$\text{final output} = \sigma(h_1.w_5 + h_2.w_6 + \text{bias})$$

$$\text{final output} = \frac{1}{1+e^{-(h_1.w_5+h_2.w_6+\text{bias})}}$$

This gives the final output of the network after applying the sigmoid activation function in output layers, introducing the desired non-linearity.

What is Artificial Neuron?

- An artificial neuron is a mathematical function that models a biological neuron's behavior.
- It takes one or more inputs, performs some computation, and produces a single output.
- The inputs and output are numbers, and the computation is a simple mathematical operation, such as a dot product followed by a non-linear function called an "activation function".



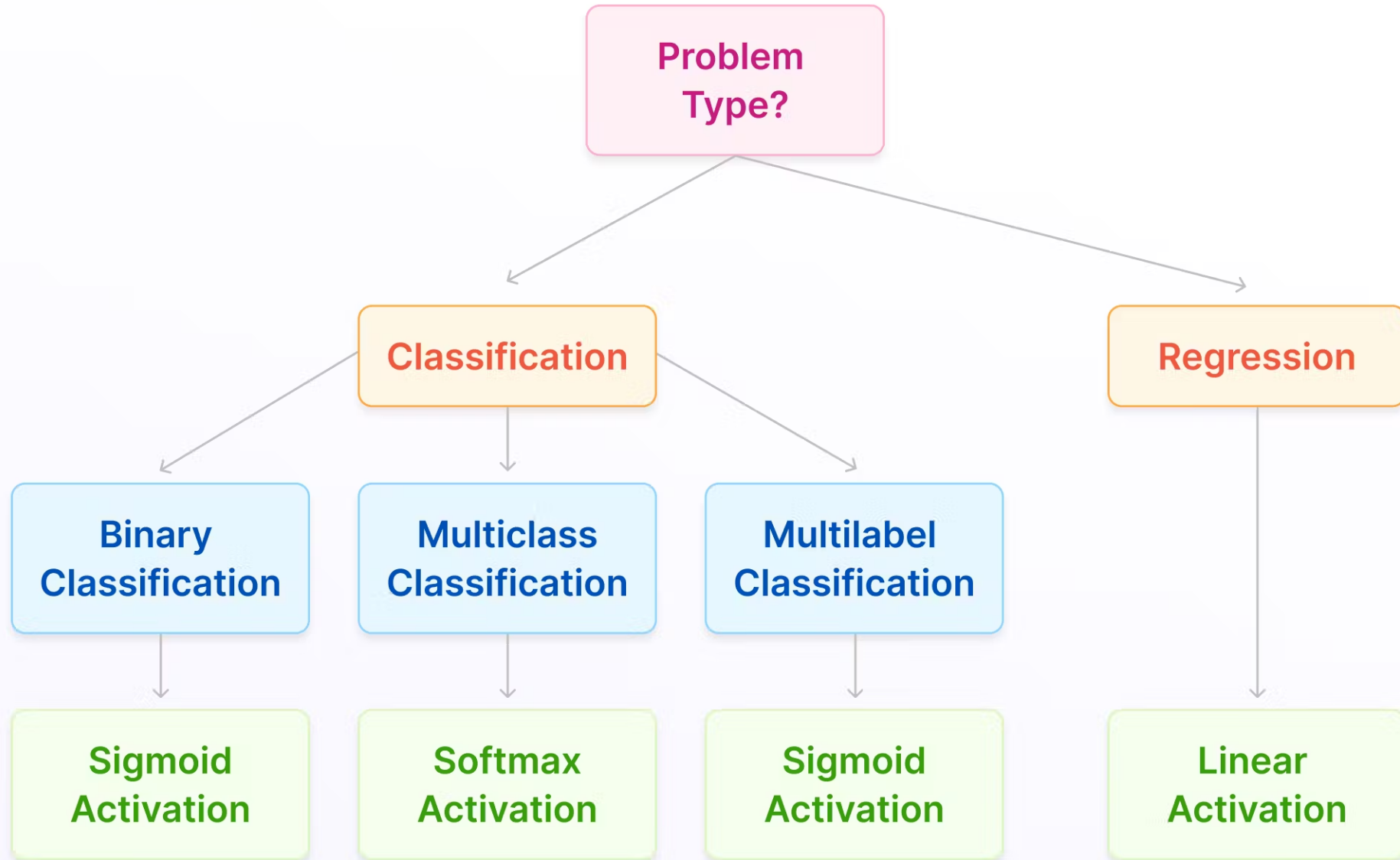
Common Activation Functions in ANNs

- Activation functions are important in neural networks because they introduce non-linearity and helps the network to learn complex patterns. Lets see some common activation functions used in ANNs:
- **Sigmoid Function:** Outputs values between 0 and 1. It is used in binary classification tasks like deciding if an image is a cat or not.
- **ReLU (Rectified Linear Unit):** A popular choice for hidden layers, it returns the input if positive and zero otherwise. It helps to solve the vanishing gradient problem.
- **Tanh (Hyperbolic Tangent):** Similar to sigmoid but outputs values between -1 and 1. It is used in hidden layers when a broader range of outputs is needed.
- **Softmax:** Converts raw outputs into probabilities used in the final layer of a network for multi-class classification tasks.
- **Leaky ReLU:** A variant of ReLU that allows small negative values for inputs helps in preventing “dead neurons” during training.
- These functions help the network decide whether to activate a neuron helps it to recognize patterns and make predictions.

Activation functions in Neural Networks

- While building a neural network, one key decision is selecting the Activation Function for both the hidden layer and the output layer.
- It is a mathematical function applied to the output of a neuron.
- It introduces non-linearity into the model, allowing the network to learn and represent complex patterns in the data.
- Without this non-linearity feature a neural network would behave like a linear regression model no matter how many layers it has.
- Activation function decides whether a neuron should be activated by calculating the weighted sum of inputs and adding a bias term.
- This helps the model make complex decisions and predictions by introducing non-linearities to the output of each neuron.

- A biological neuron receives input signals through dendrites, and a perceptron receives data through input neurons
- The connections between dendrites and biological neurons are called synapses, and the connections between inputs and perceptrons are called weights
- The nucleus in a biological neuron produces an output signal based on the input signals, and the nucleus in a perceptron performs calculations to produce an output
- In a biological neuron, the output signal is carried away by the axon, and in a perceptron, the output value is used as input for the next perceptron.



Used in layer	Activation Function	Details	Pros	Cons
Hidden / Output	Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$ Output range: [0,1]	- Smooth activation that outputs values between 0 and 1, making it suitable for binary classification - Historically popular	- Vanishing gradient problem due to saturated neurons - Output not zero-centered as sigmoid outputs are always positive - exp() operation is computationally intensive
Hidden	Tanh (Hyperbolic tangent)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Output range: [-1,1]	- Zero centered outputs that help networks train faster	- Suffers with vanishing gradient problem when saturated
Hidden	ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$ Output range: [0, ∞)	- Does not saturate: avoids vanishing gradient issues for positive inputs - Computationally efficient since only certain number of neurons are activated at the same time - Much faster convergence compared to sigmoid/tanh	- Output not zero-centered - Prone to a "dying ReLU" problem, where neurons can get stuck during training and never activate again, leading to a dead neuron that doesn't update its weights. © AIML.com Research

Used in layer	Activation Function	Details	Pros	Cons
Hidden	Leaky ReLU	$f(x) = \max(0.01x, x)$ Output range: $(-\infty, \infty)$	- All benefits of ReLU - Addresses the "dying ReLU" problem by allowing a small gradient for negative inputs. Helps with training deeper networks	- Not as standardized as ReLU, and the slope of the leaky part is typically a hyperparameter (α) that needs tuning. This is also referred to as Parametric ReLU $f(x) = \max(\alpha x, x)$
Hidden	ELU (Exponential Linear Unit)	where $\alpha > 0$: $f(x) = \begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$ α is commonly chosen as 1 Output range: $(-\alpha, \infty)$	- All benefits of ReLU - Zero centered outputs that help networks train faster - ELUs saturate to a negative value when the argument gets smaller becoming more robust to noise	- Computationally more expensive due to exponential operation
Output	Softmax	For a given class i, probability $P(y_i)$ is: $P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$ where, z_i is the raw score (logit) for class i, and N is the no. of classes Output range: $(0,1)$	- Used in the output layer of multi-class classification problems, converting model outputs into probability distributions.	- Not suitable for multi-label classification, as it enforces that only one class can be predicted. © AIML.com Research

Biological Neuron vs. Artificial Neuron

Biological Neuron	Artificial Neuron
Biological neurons are cells in living organisms that process and transmit information.	Artificial neurons are computer programs that mimic the behavior of biological neurons and are the building blocks of artificial neural networks (ANNs).
Biological neurons use electrochemical signals.	Artificial neurons use digital signals.
Biological neurons take longer to process information compared to artificial neurons.	Artificial neural networks can process information more quickly than biological neurons.
Larger in size.	Smaller in size.

What are the commonly used activation functions?

- Some of the commonly used activation functions are **binary, sigmoidal (linear), and tan hyperbolic sigmoidal functions(nonlinear)**.
- **Binary** - The output has only two values, either 0 or 1. For this, the threshold value is set up. If the net weighted input is greater than 1, the output is assumed as one; otherwise, it is zero.
- **Sigmoidal Hyperbolic** - This function has an 'S' shaped curve. Here, the tan hyperbolic function is used to approximate the output of the net input. The function is defined as - $f(x) = \frac{1}{1 + \exp(-\alpha x)}$ where α - steepness parameter.

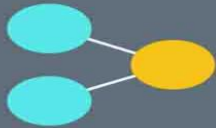
Types of Neural Networks

Parameter	Types	Description
Based on the connection pattern	FeedForward, Recurrent	Feedforward - In which graphs have no loops. Recurrent - Loops occur because of feedback.
Based on the number of hidden layers	Single-layer, Multi-Layer	Single Layer - Having one secret layer. E.g., Single Perceptron Multilayer - Having multiple secret layers. Multilayer Perceptron
Based on the nature of weights	Fixed, Adaptive	Fixed - Weights are a fixed priority and not changed at all. Adaptive - Updates the weights and changes during training.
Based on the Memory unit	Static, Dynamic	Static - Memoryless unit. The current output depends on the current input. E.g., Feedforward network. Dynamic - Memory unit - The output depends upon the current input as well as the current output. E.g., Recurrent Neural Network

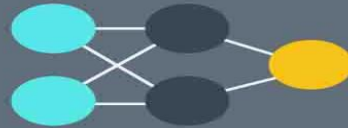
Neural Network Architecture Types

1. Perceptron Model in Neural Networks
2. Radial Basis Function Neural Network
3. Multilayer Perceptron Neural Network
4. Recurrent Neural Network
5. Long Short-Term Memory Neural Network (LSTM)
6. Hopfield Network
7. Boltzmann Machine Neural Network
8. Convolutional Neural Network
9. Modular Neural Network
10. Physical Neural Network

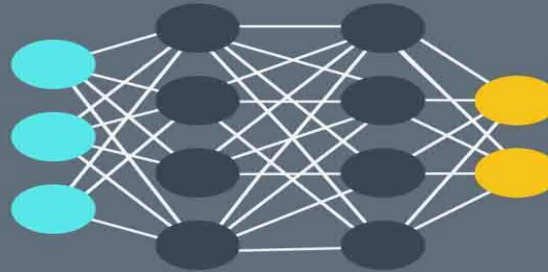
NEURAL NETWORK ARCHITECTURE TYPES



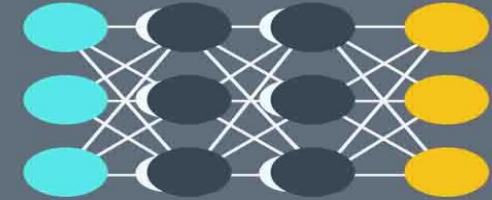
SINGLE LAYER PERCEPTRON



RADIAL BASIS NETWORK



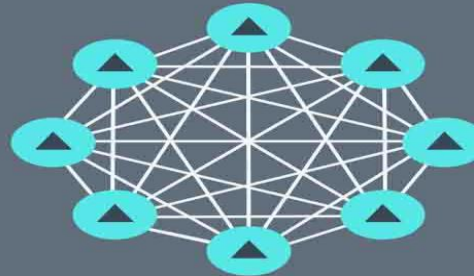
MULTI LAYER PERCEPTRON



RECURRENT NEURAL NETWORK



LSTM RECURRENT NEURAL NETWORK



HOPFIELD NETWORK



BOLTZMANN MACHINE



INPUT UNIT



OUTPUT UNIT



HIDDEN UNIT



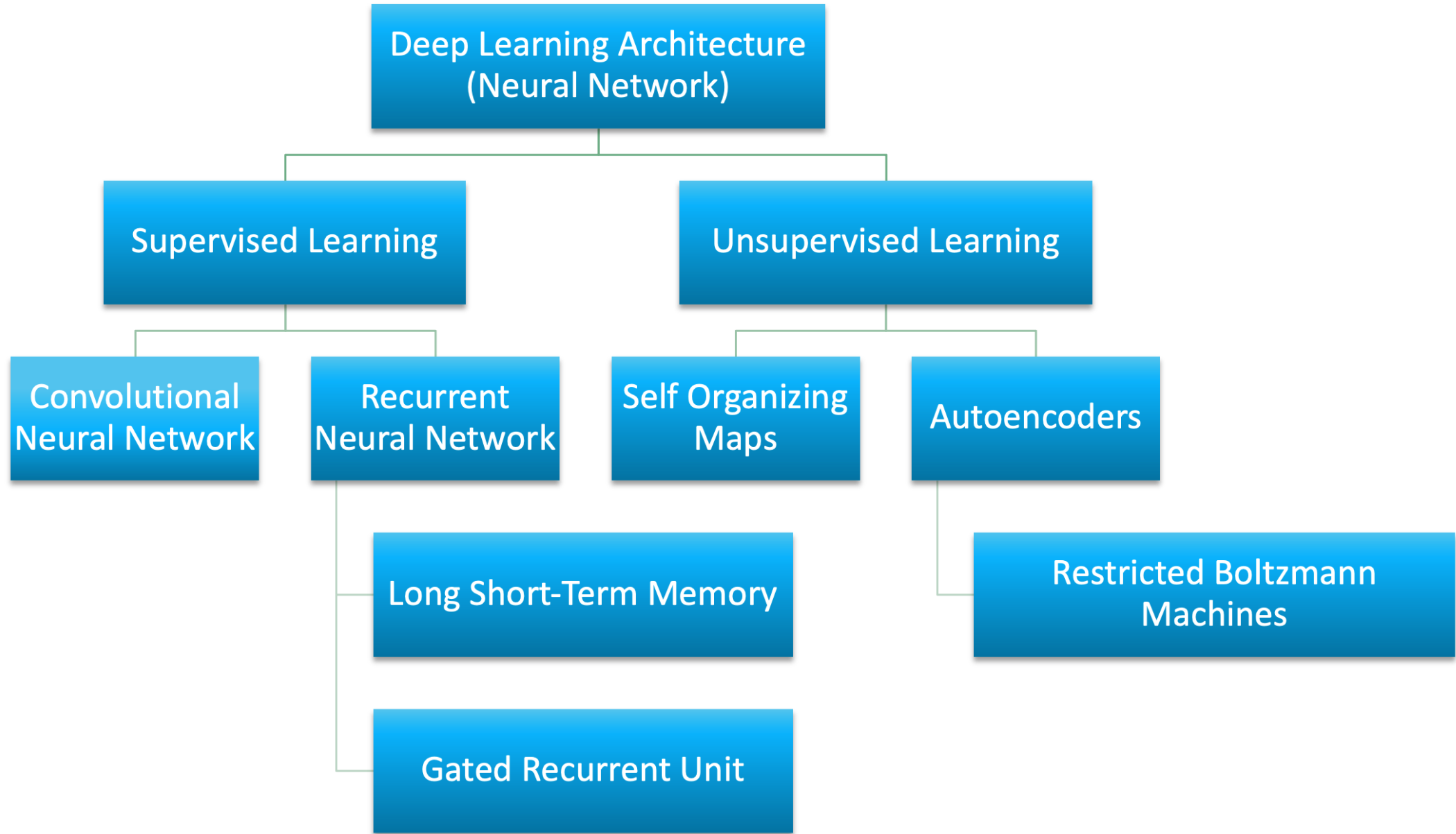
FEEDBACK WITH MEMORY UNIT



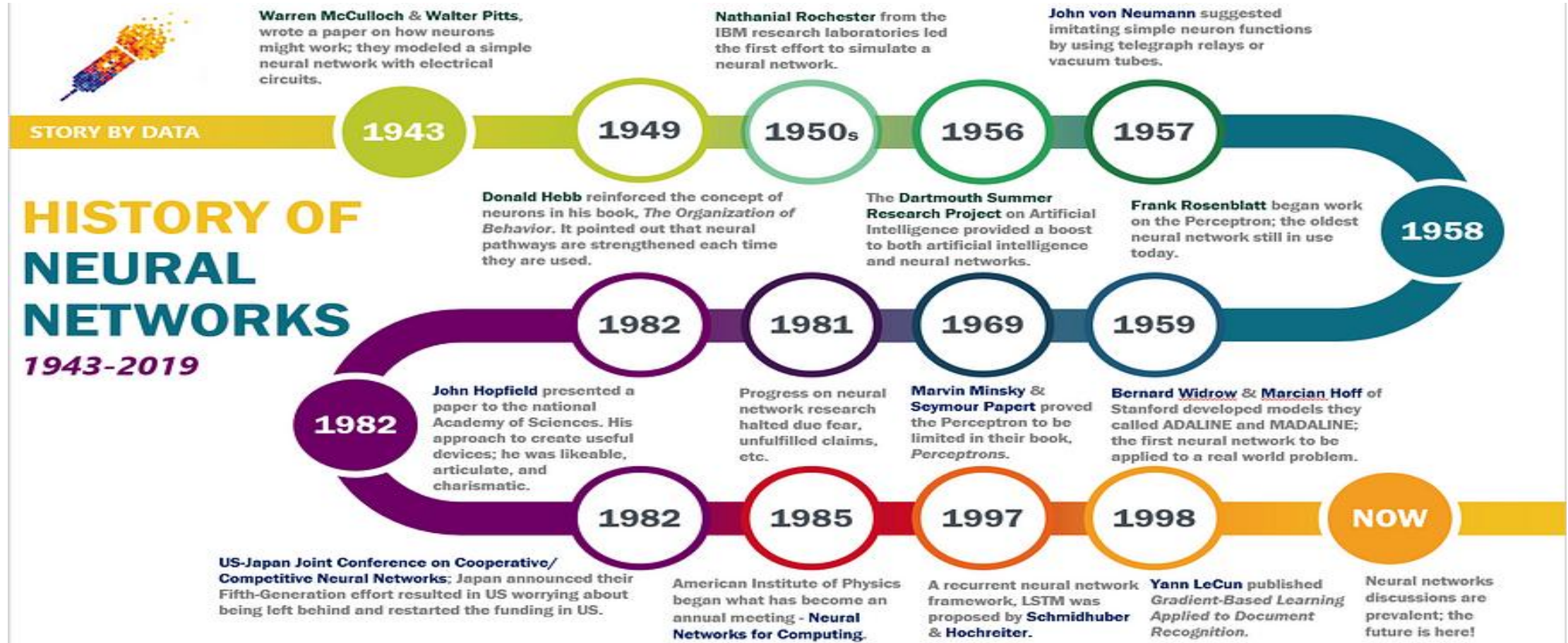
BACKFED INPUT UNIT



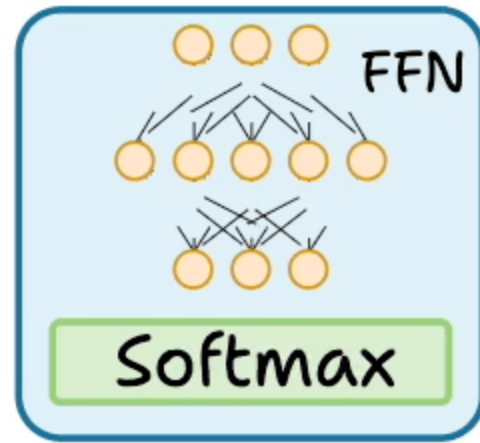
PROBABILISTIC HIDDEN UNIT



History of Artificial Neural Network



Router



softmax scores
over experts

selected_expert _ _ _



Expert-1

Expert-2

Expert-3

Expert-4

- **Learning Processes:** Learning Processes: Introduction, Error correction learning, Memory-based learning, Hebbian learning, Competitive learning, Boltzmann learning, credit assignment problem, Learning with and without teacher, learning tasks, Memory and Adaptation.

What are the Learning Rules in Neural Network?

- **Learning rule** or **Learning process** is a method or a mathematical logic.
- It improves the **Artificial Neural Network's** performance and applies this rule over the network.
- Thus learning rules updates the weights and bias levels of a network when a network simulates in a specific data environment.
- Applying learning rule is an iterative process.
- It helps a neural network to learn from the existing conditions and improve its performance.

Hebbian Learning Rule

01

Perceptron Learning Rule

02

Delta Learning Rule

03

Correlation Learning Rule

04

Out Star Learning Rule

05

Neural Network Rules

Different learning rules in the Neural network:

- **Hebbian learning rule** – It identifies, how to modify the weights of nodes of a network.
- **Perceptron learning rule** – Network starts its learning by assigning a random value to each weight.
- **Delta learning rule** – Modification in synaptic weight of a node is equal to the multiplication of error and the input.
- **Correlation learning rule** – The correlation rule is the supervised learning.
- **Outstar learning rule** – We can use it when it assumes that nodes or neurons in a network arranged in a layer.

HEBBIAN LEARNING RULE



"Neurons that fire together wire together"

PERCEPTRON LEARNING RULE

$$w_2(\eta) = w_2(o) + \eta \cdot (t - o) \cdot x_i$$



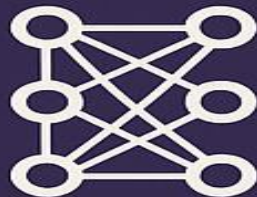
$w_1 = n$ singlelayer perceptron

DELTA RULE (Widrow-Hoff Rule)



Based on gradient descent

BACKPROPAGATION ALGORITHM



Used in multilayer perceptrons

COMPETITIVE LEARNING RULE



$\Delta w_i = \eta \cdot (x - w)$
Neurons compete and update weights

REINFORCEMENT LEARNING RULE



Learning through rewards or penalties

Hebbian Learning Rule – Key Points

1. Developed by Donald Hebb in 1949

1. First learning rule in neural networks.
2. Designed for **unsupervised learning**.

2. Basic Principle

1. *"Neurons that fire together, wire together."*

3. Weight Update Logic

1. If **two neighboring neurons** activate/deactivate **together**:
 1. **Increase** the weight between them.
2. If neurons activate in **opposite phase**:
 1. **Decrease** the weight.
3. If there's **no correlation** in activity:
 1. **No change** in the weight.

$$W_{ij} = x_i * x_j$$

4. Input Sign Relation

1. Both inputs positive or both negative → **strong positive weight**.
2. One input positive, other negative → **strong negative weight**.

5. Initialization

1. All weights start at **zero**.

6. Activation Compatibility

1. Works with **soft and hard activation functions**.

7. Unsupervised Nature

1. No target output is needed.
2. Learning is based purely on **input correlations**.

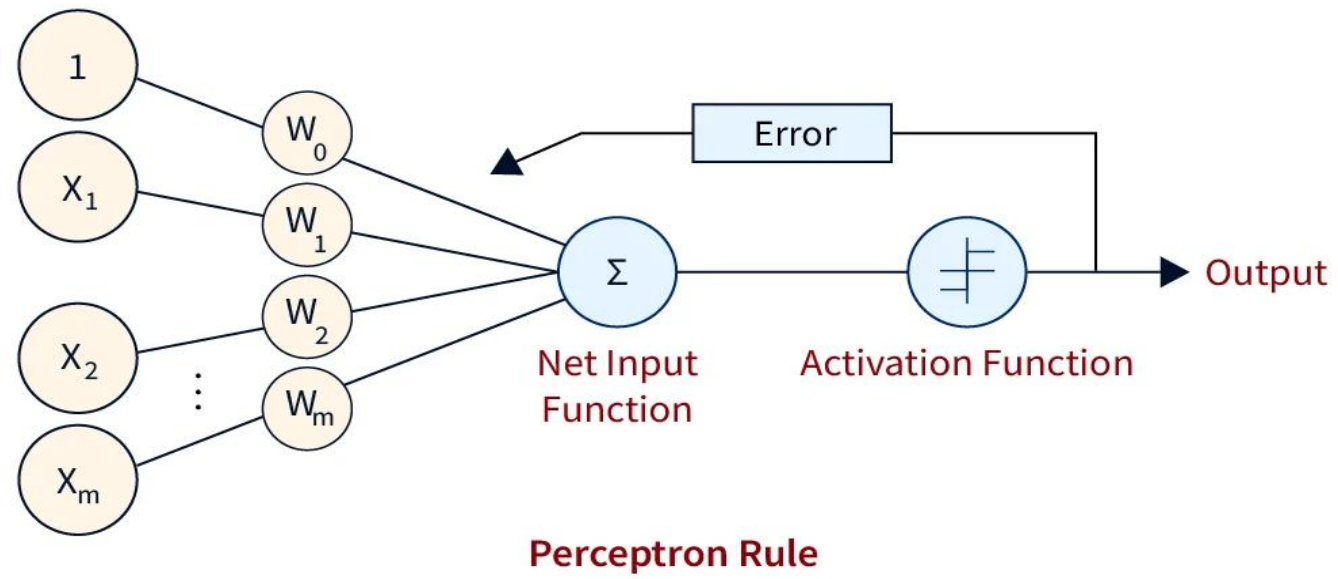
8. Limitations

1. Weight magnitudes often grow with **learning time** (undesired effect).

What is the Perceptron Learning Algorithm?

There are four significant steps in a perceptron learning algorithm:

1. First, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:
$$\sum w_i * x_i = x_1 * w_1 + x_2 * w_2 + \dots + w_n * x_n.$$
 Add another essential term called bias 'b' to the weighted sum to improve the model performance. $\sum w_i * x_i + b.$
2. Next, an activation function is applied to this weighed sum, producing a binary or a continuous-valued output. $Y = f(\sum w_i * x_i + b)$
3. Next, the difference between this output and the actual target value is computed to get the error term, **E**, generally in terms of mean squared error. The steps up to this form the forward propagation part of the algorithm. $E = (Y - Y_{actual})^2$
4. We optimize this error (loss function) using an optimization algorithm. Generally, some form of gradient descent algorithm is used to find the optimal values of the hyperparameters like **learning rate**, **weight**, **Bias**, etc. This step forms the backward propagation part of the algorithm.



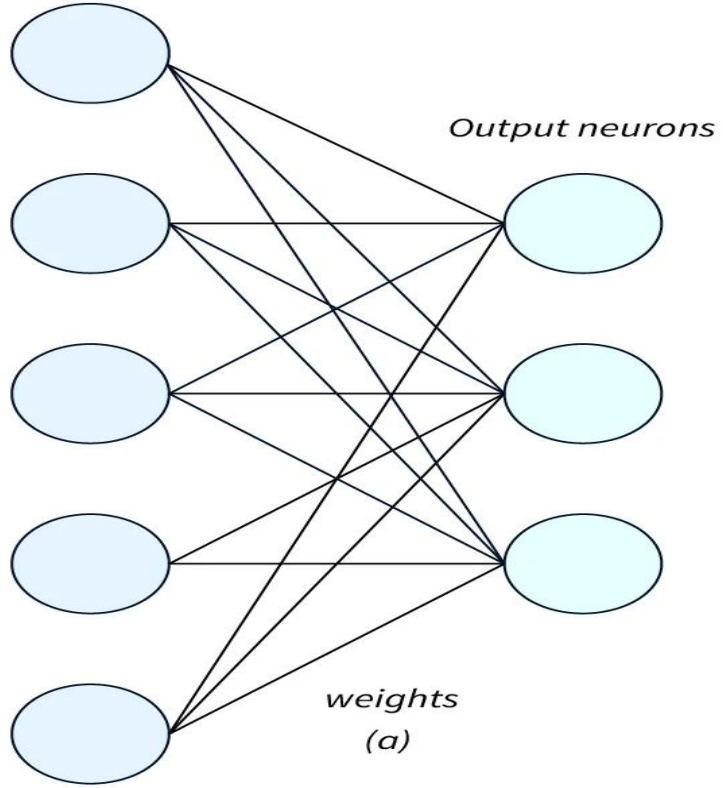
Single Layer Perceptron Model:

- It is the simplest Artificial Neural Network (ANN) model.
- A single-layer perceptron model consists of a feed-forward network and includes a threshold transfer function for thresholding on the Output.
- The main objective of the single-layer perceptron model is to classify linearly separable data with binary labels.

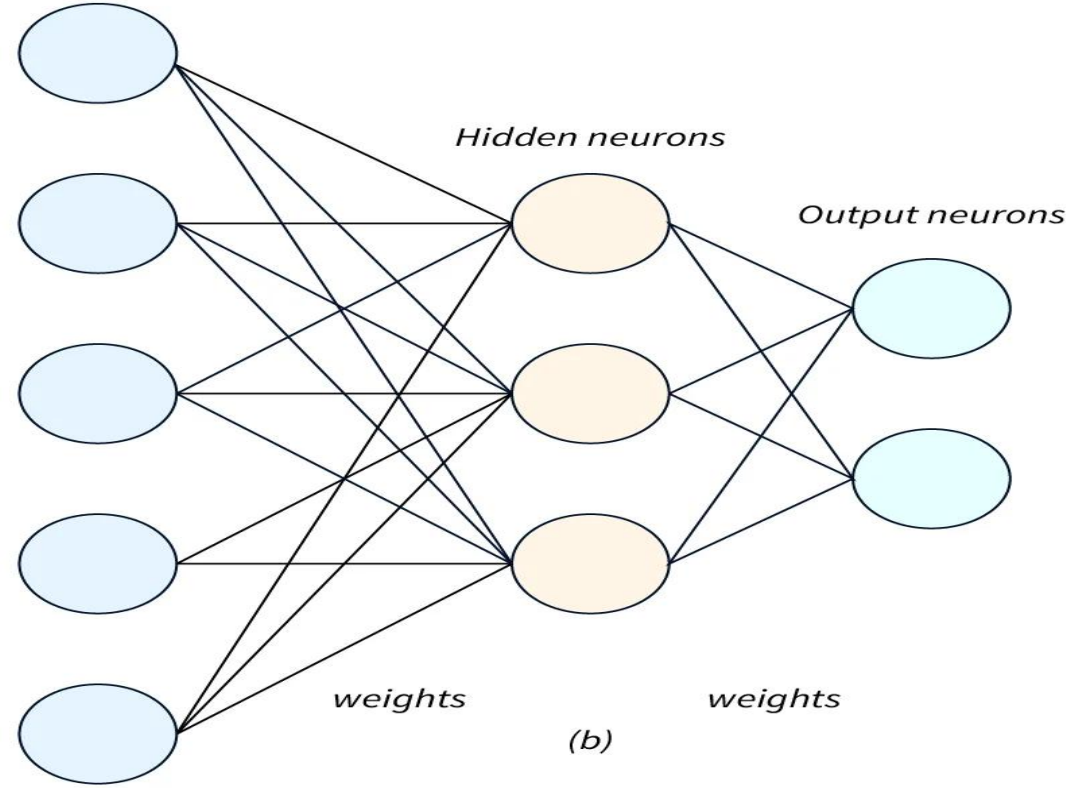
Multi-Layer Perceptron Model:

- The multi-layer perceptron learning algorithm has the same structure as a single-layer perceptron but consists of an additional one or more hidden layers, unlike a single-layer perceptron, which consists of a single hidden layer.

Input neurons



Input neurons



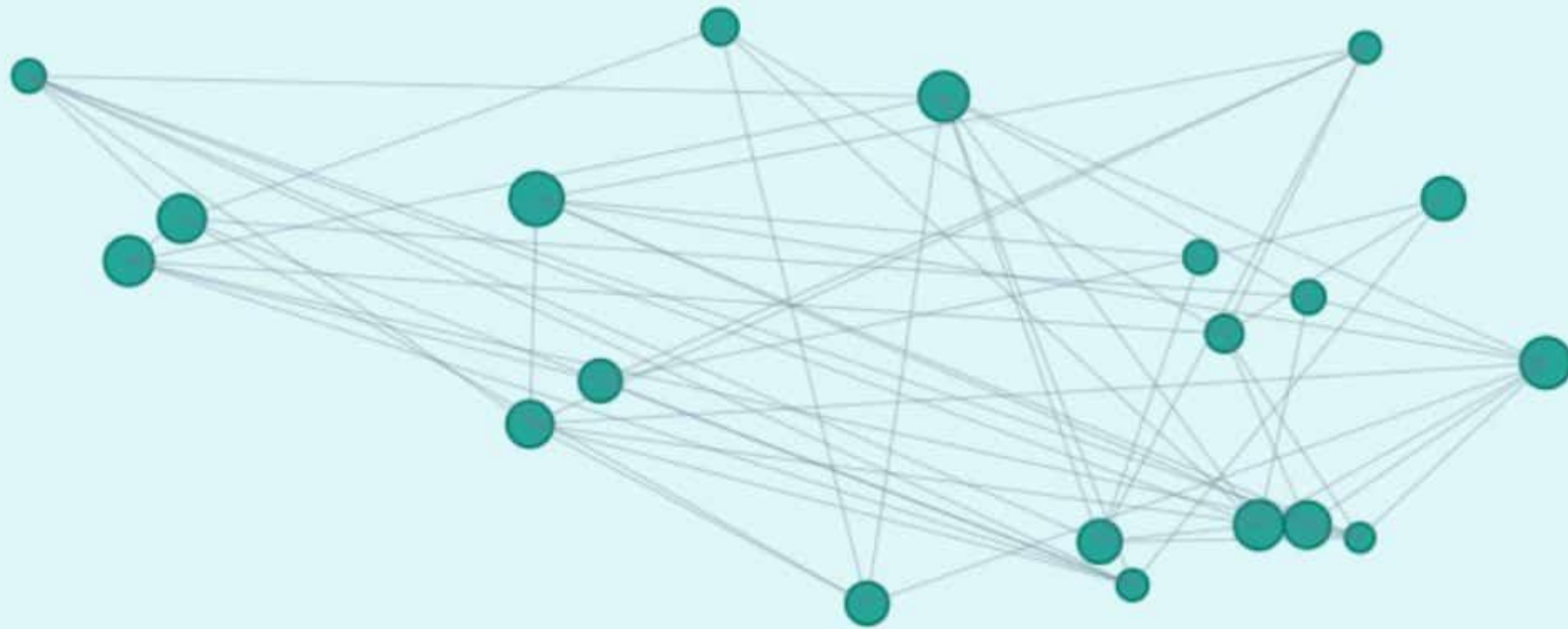
(a) Architecture of a single layer perceptron. The architecture consists of a layer on input neurons fully connected to a single layer of output neurons.

(b) Extension to a multi-layer perceptron including more than one layer of trainable weights. In this example, the network includes 3 layers: input, hidden and output layer. Each connection between two neurons is given by a certain weight.

Boltzmann Machine

- Boltzmann Machines (BMs) are neural networks used in unsupervised learning, optimization, and feature extraction.
- This article covers their working principles, types like RBMs, DBMs, and Full BMs, key applications in image recognition, NLP, recommendation systems, and more, along with challenges and future advancements.

Boltzmann Machine

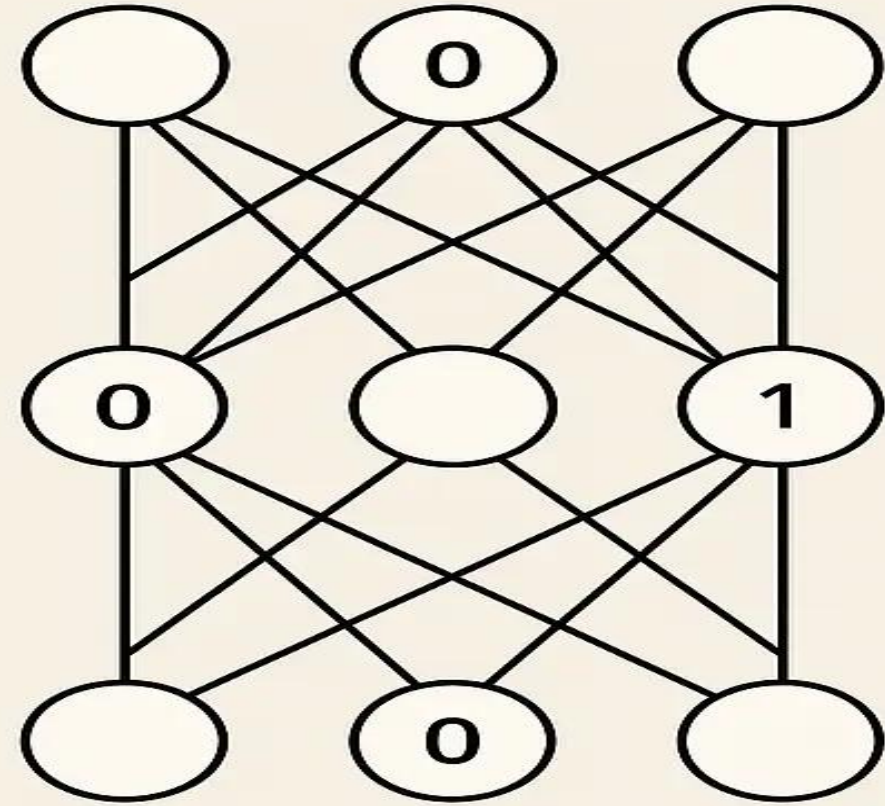


What is a Boltzmann Machine?

A Boltzmann Machine is a type of stochastic recurrent neural network that uses a probabilistic approach to model data distributions.

It consists of a set of neurons connected in a network, where each neuron can take binary states (0 or 1).

These networks are energy-based models that learn by minimizing an energy function, similar to how physical systems reach equilibrium.



How Does a Boltzmann Machine Work?

- **Energy Function Calculation:** The energy of a given state is computed using weights and biases associated with the neurons.
- **Gibbs Sampling:** Neurons are updated one by one using a probability distribution derived from the energy function.
- **Training via Contrastive Divergence:** The network learns optimal weights through iterative updates that minimize the difference between data-driven and model-driven distributions.
- **Convergence to Equilibrium:** Over multiple iterations, the network adjusts its parameters to model the underlying data distribution accurately.

Types of Boltzmann Machines

Types of Boltzmann Machines

Restricted Boltzmann Machine

Efficient learning with two-layer architecture



Deep Boltzmann Machine

Multi-layer representation with bidirectional connections



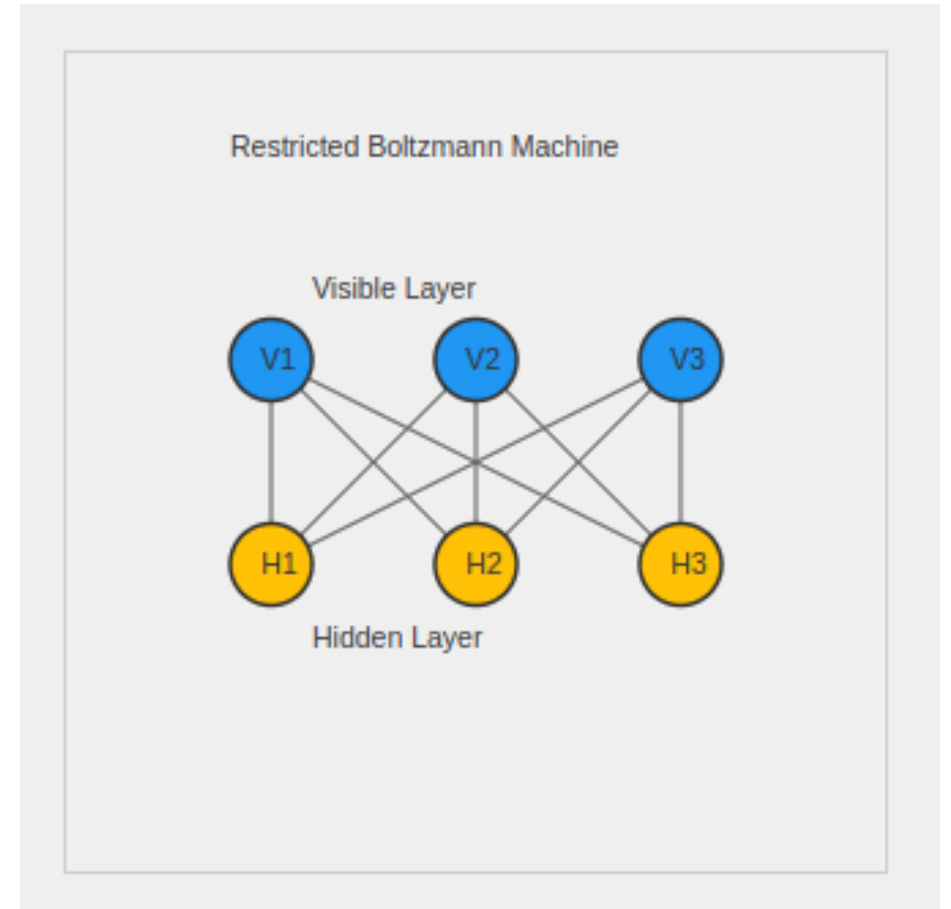
Full Boltzmann Machine

Highly expressive but computationally expensive



1. Restricted Boltzmann Machine (RBM)

- Two-layer architecture:** RBMs consist of one visible layer (input) and one hidden layer (feature detectors). There are no connections between neurons in the same layer.
- Efficient learning:** The absence of intra-layer connections allows for faster and more efficient learning compared to full Boltzmann Machines.
- Training via Contrastive Divergence:** RBMs are trained using an approximation method called Contrastive Divergence (CD), which helps reduce computational cost.
- Common applications:** Used in feature learning, dimensionality reduction, collaborative filtering (e.g., Netflix recommendation system), and pretraining deep networks like Deep Belief Networks (DBNs).



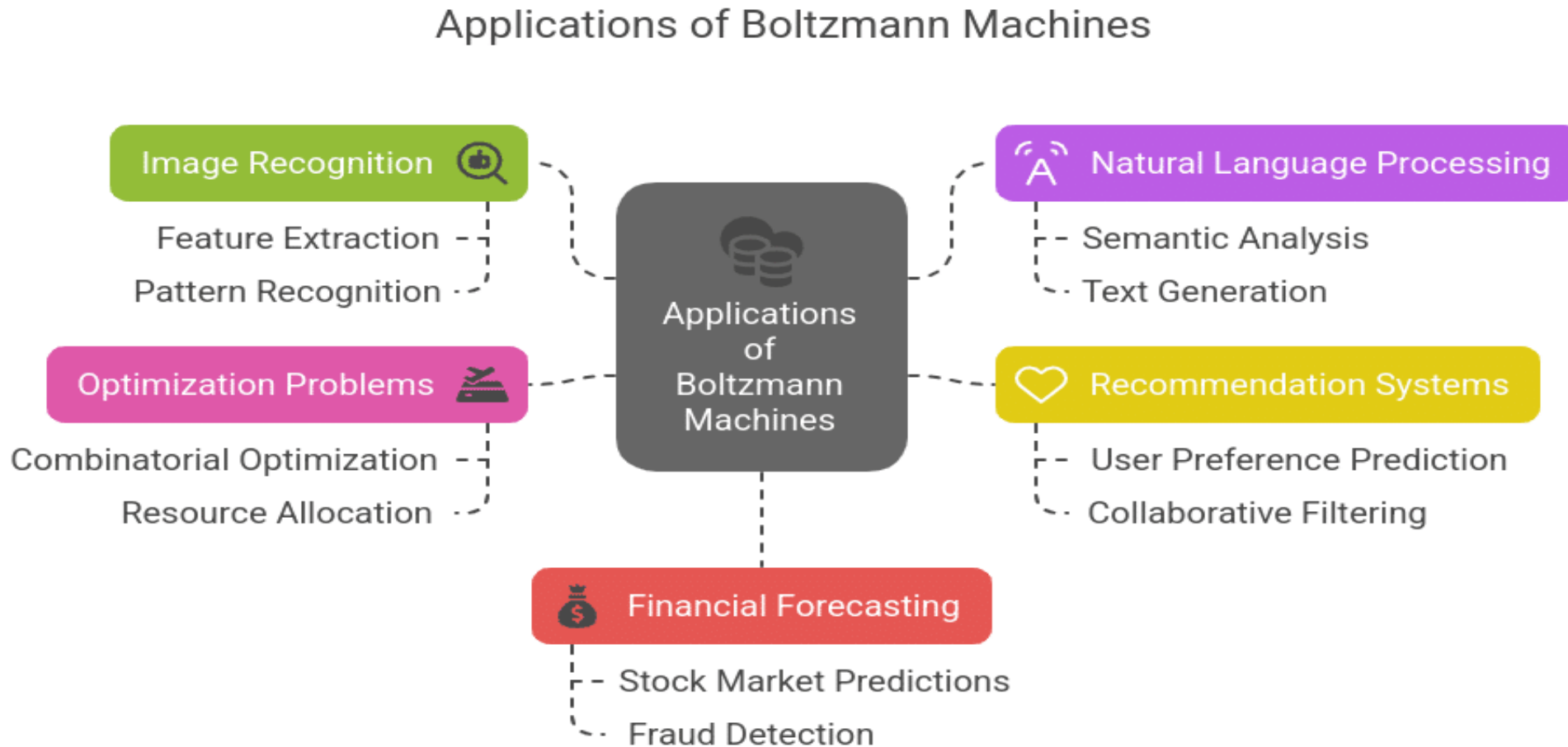
2. Deep Boltzmann Machine (DBM)

- **Hierarchical feature learning:** DBMs learn multi-layer representations, capturing complex patterns in data.
- **Bidirectional connections:** Unlike Deep Belief Networks (DBNs), where connections are strictly directed, DBMs have bidirectional connections between layers, allowing better generative modeling.
- **Improved performance:** By stacking RBMs, DBMs can model high-level abstractions and learn more meaningful representations.
- **Training complexity:** Training DBMs is more computationally intensive than RBMs, often requiring layer-wise pretraining followed by [fine-tuning](#) using approximate inference techniques.
- **Applications:** Used in [unsupervised learning](#), image and [speech recognition](#), and deep generative modeling.

3.Full Boltzmann Machine

- **Fully connected architecture:** Every neuron is connected to every other neuron, making the network highly expressive but computationally expensive.
- **Powerful representation capacity:** Due to their unrestricted connectivity, full Boltzmann Machines can model complex probability distributions.
- **Slow training process:** The high number of connections makes training difficult, as Gibbs Sampling becomes impractical for large networks.
- **Limited practical use:** Because of their supreme computational cost, full Boltzmann Machines have had almost no practical use and have mainly been employed in small applications and theoretical studies.

Applications of Boltzmann Machines



Fundamentals of Pattern Recognition & Perceptron Models

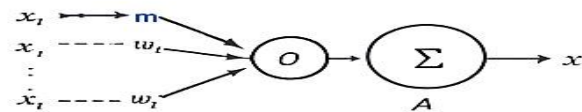
Introduction to Liteair
Classifiers and Perceptron Algorithms

Core Elements of Pattern Recognition



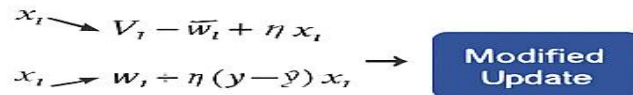
The process includes data acquisition, preprocessing, classification, and decision-making.

The Simple Perceptron Model



A perceptron computes a weighted sum of inputs and applies an activation function.

Adaptive Perceptron Algorithm



Adjustments improve learning speed and handle linearly separable classes more

Learning Mechanism in Continuous Models



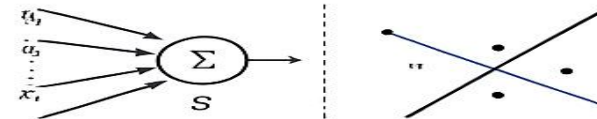
Gradient-based optimization is used to minimize a differentiable loss function:

What is Pattern Recognition?



Pattern recognition involves classifying input data into categories based on feature extractors. Usually used in AI, vision, speech, and medical diagnostics.

Core Elements of Pattern Recognition



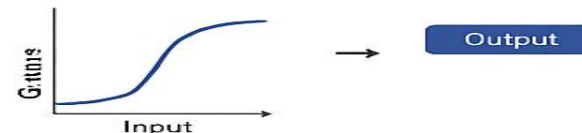
A linear-classifier separates data points using a decision boundary. Used in early AI systems.

Learning with Perceptron

Original Rule	Modified (perceptron) rule
Original update	$w_i \leftarrow w_i + \eta(y - x_i)$
For each training example.	$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$

The algorithm updates weights based on classification error until convergence.

Modified Perceptron Algorithm



A continuous perceptron allows smoother learning and better generalization.

Learning with Continuous Models



Cannot solve non-linearly separable problems (e.g., XOR). Limited to linear boundaries.

Single layer Perception

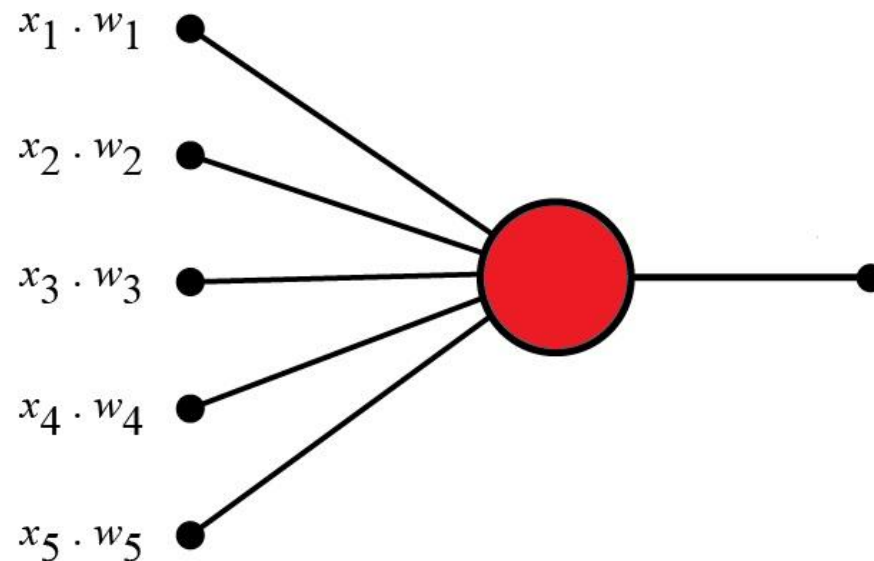
- A **Perceptron** is an **Artificial Neuron**.
- It is the simplest possible **Neural Network**.
- **Neural Networks** are the building blocks of **Machine Learning**.

Single layer Perception

- **Frank Rosenblatt** (1928 – 1971) was an American psychologist notable in the field of Artificial Intelligence.
- In **1957** he started something really big. He "invented" a **Perceptron** program, on an IBM 704 computer at Cornell Aeronautical Laboratory.
- Scientists had discovered that brain cells (**Neurons**) receive input from our senses by electrical signals.
- The Neurons, then again, use electrical signals to store information, and to make decisions based on previous input.
- Frank had the idea that **Perceptrons** could simulate brain principles, with the ability to learn and make decisions.

The Perceptron

- The original **Perceptron** was designed to take a number of **binary** inputs, and produce one **binary** output (0 or 1).
- The idea was to use different **weights** to represent the importance of each **input**, and that the sum of the values should be greater than a **threshold** value before making a decision like **yes** or **no** (true or false) (0 or 1).



Perceptron Example

- Imagine a perceptron (in your brain).
- The perceptron tries to decide if you should go to a concert.
- Is the artist good? Is the weather good?
- What weights should these facts have?

Criteria	Input	Weight
Artists is Good	$x_1 = 0 \text{ or } 1$	$w_1 = 0.7$
Weather is Good	$x_2 = 0 \text{ or } 1$	$w_2 = 0.6$
Friend will Come	$x_3 = 0 \text{ or } 1$	$w_3 = 0.5$
Food is Served	$x_4 = 0 \text{ or } 1$	$w_4 = 0.3$
Water is Served	$x_5 = 0 \text{ or } 1$	$w_5 = 0.4$

The Perceptron Algorithm

- Frank Rosenblatt suggested this algorithm:
- Set a threshold value
- Multiply all inputs with its weights
- Sum all the results
- Activate the output

- **1. Set a threshold value:**
 - Threshold = 1.5
- **2. Multiply all inputs with its weights:**
 - $x_1 * w_1 = 1 * 0.7 = 0.7$
 - $x_2 * w_2 = 0 * 0.6 = 0$
 - $x_3 * w_3 = 1 * 0.5 = 0.5$
 - $x_4 * w_4 = 0 * 0.3 = 0$
 - $x_5 * w_5 = 1 * 0.4 = 0.4$
- **3. Sum all the results:**
 - $0.7 + 0 + 0.5 + 0 + 0.4 = 1.6$ (The Weighted Sum)
- **4. Activate the Output:**
 - Return true if the sum > 1.5 ("Yes I will go to the Concert")
 - $0.7 > 1.5$ (FALSE)

- Perceptron in AI
- A **Perceptron** is an **Artificial Neuron**.
- It is inspired by the function of a **Biological Neuron**.
- It plays a crucial role in **Artificial Intelligence**.
- It is an important building block in **Neural Networks**.
- To understand the theory behind it, we can break down its components:
- Perceptron Inputs (nodes)
- Node values (1, 0, 1, 0, 1)
- Node Weights (0.7, 0.6, 0.5, 0.3, 0.4)
- Summation
- Treshold Value
- Activation Function
- Summation ($\text{sum} > \text{treshold}$)

1. Perceptron Inputs

- A perceptron receives one or more input.
- Perceptron inputs are called **nodes**.
- The nodes have both a **value** and a **weight**.

2. Node Values (Input Values)

- Input nodes have a binary value of **1** or **0**.
- This can be interpreted as **true** or **false** / **yes** or **no**.
- The values are: 1, 0, 1, 0, 1

3. Node Weights

- Weights are values assigned to each input.
- Weights shows the **strength** of each node.
- A higher value means that the input has a stronger influence on the output.
- The weights are: 0.7, 0.6, 0.5, 0.3, 0.4

4. Summation

- The perceptron calculates the weighted sum of its inputs.
- It multiplies each input by its corresponding weight and sums up the results.
- The sum is: $0.7*1 + 0.6*0 + 0.5*1 + 0.3*0 + 0.4*1 = 1.6$

6. The Threshold

The Threshold is the value needed for the perceptron to fire (outputs 1), otherwise it remains inactive (outputs 0).

In the example, the threshold value is: 1.5

7. The Activation Function

After the summation, the perceptron applies the activation function.

The purpose is to introduce non-linearity into the output. It determines whether the perceptron should fire or not based on the aggregated input.

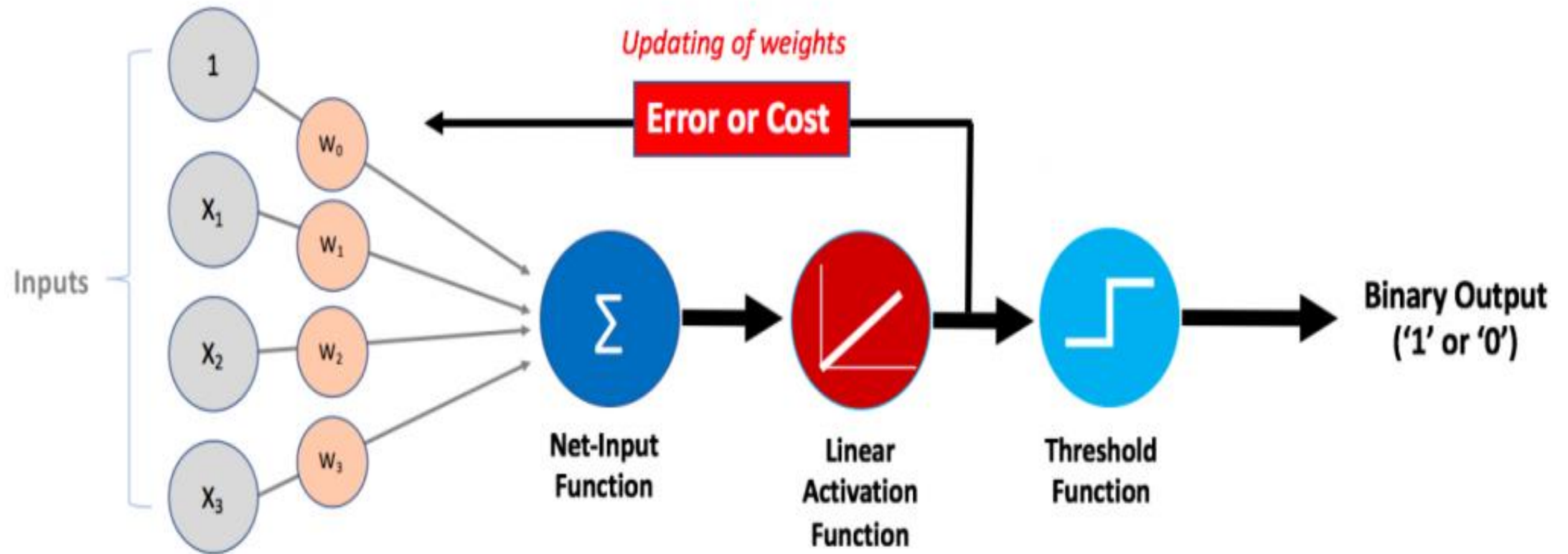
The activation function is simple: $(\text{sum} > \text{threshold}) == (1.6 > 1.5)$

Criteria	Input	Weight
Artists is Good	$x_1 = 1$	$w_1 = 1.4$
Weather is Good	$x_2 = 0$	$w_2 = 0.1$
Friend will Come	$x_3 = 1$	$w_3 = -1.5$
Food is Served	$x_4 = 0$	$w_4 = 0.7$
Water is Served	$x_5 = 1$	$w_5 = 0.1$

Classification	Clustering
Uses labelled data as the input	Uses unlabelled data as the input
The output is known	The output is unknown
Uses supervised machine learning	Uses unsupervised machine learning
A training data set is provided and used to produce classifications	A training data set is not provided and used to produce clusters
Examples of algorithms: Decision-trees, Bayesian Classifiers and Support Vector Machines (SVM)	Examples of algorithms: Partition-based clustering (k-means), Hierarchical clustering (agglomerative & divisive) and DBSCAN
Can be more complex than clustering	Can be less complex than classification
Does not specify areas for improvement	Specifies areas for improvement
Two-phase	Single-phase
Boundary conditions must be specified	Boundary conditions do not always need to be specified

Adaptive linear combiner

- An adaptive linear neuron, or [ADALINE](#), is a single-layer neural network with a linear activation function that learns from data by adjusting its weights and biases using the Delta Rule or Least Mean Square (LMS) algorithm to minimize squared error.
- It was developed by Bernard Widrow and Ted Hoff in 1960 and is used for [linear regression](#) and [binary classification](#), serving as a basic component in more complex neural networks.



Net Input function

- **Combination of Input signals of different strength (weights):**
- Input signals of different strength (weights) get combined / added in order to be fed into the activation function.
- The combined input or sum of weighted inputs can also be called as **net input**.

- **Net input is fed into activation function (Linear):**
- Net input is fed into activation function.
- The activation function of adaline is an identity function.
- If Z is net input, the identity function would look like $g(Z) = Z$.
- The activation function is linear activation function as the output of the function is linear combination of input signals and weights.

Activation function output is used to learn weights:

- The output of activation function (same as net input owing to identity function) is used to calculate the change in weights related to different inputs which will be updated to learn new weights.
- Pay attention to feedback loop shown with text **Error or cost**.
- Recall that in Perceptron, the activation function is a unit step function and the output is binary (1 or 0) based on whether the net input value is greater than or equal to zero (0) or otherwise.

Threshold function – Binary prediction (1 or 0) based on unit step function:

- The prediction made by Adaline neuron is done in the same manner as in case of Perceptron.
- The output of activation function, which is net input is compared with 0 and the output is 1 or 0 depending upon whether the net input is greater than or equal to 0.
- Pay attention the diagram as to how the output of activation function is fed into threshold function.

Adaptive Linear Neuron Learning algorithm

Step 0: initialize the weights and the bias are set to some random values but not to zero, also initialize the learning rate α .

Step 1 – perform steps 2-7 when stopping condition is false.

Step 2 – perform steps 3-5 for each bipolar training pair $s:t$.

Step 3 – Activate each input unit as follows –

$$x_i = s_i (i = 1 \text{ to } n)$$

Step 4 – Obtain the net input with the following relation –

$$y_{in} = \sum_i^n x_i \cdot w_i + b$$

Here 'b' is bias and 'n' is the total number of input neurons.

Step 5 Until least mean square is obtained ($t - y_{in}$), Adjust the weight and bias as follows –

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_{in})x_i$$

$$b(\text{new}) = b(\text{old}) + \alpha(t - y_{in})$$

Now calculate the error using $\Rightarrow E = (t - y_{in})^2$

Step 7 – Test for the stopping condition, if error generated is less then or equal to specified tolerance then stop.

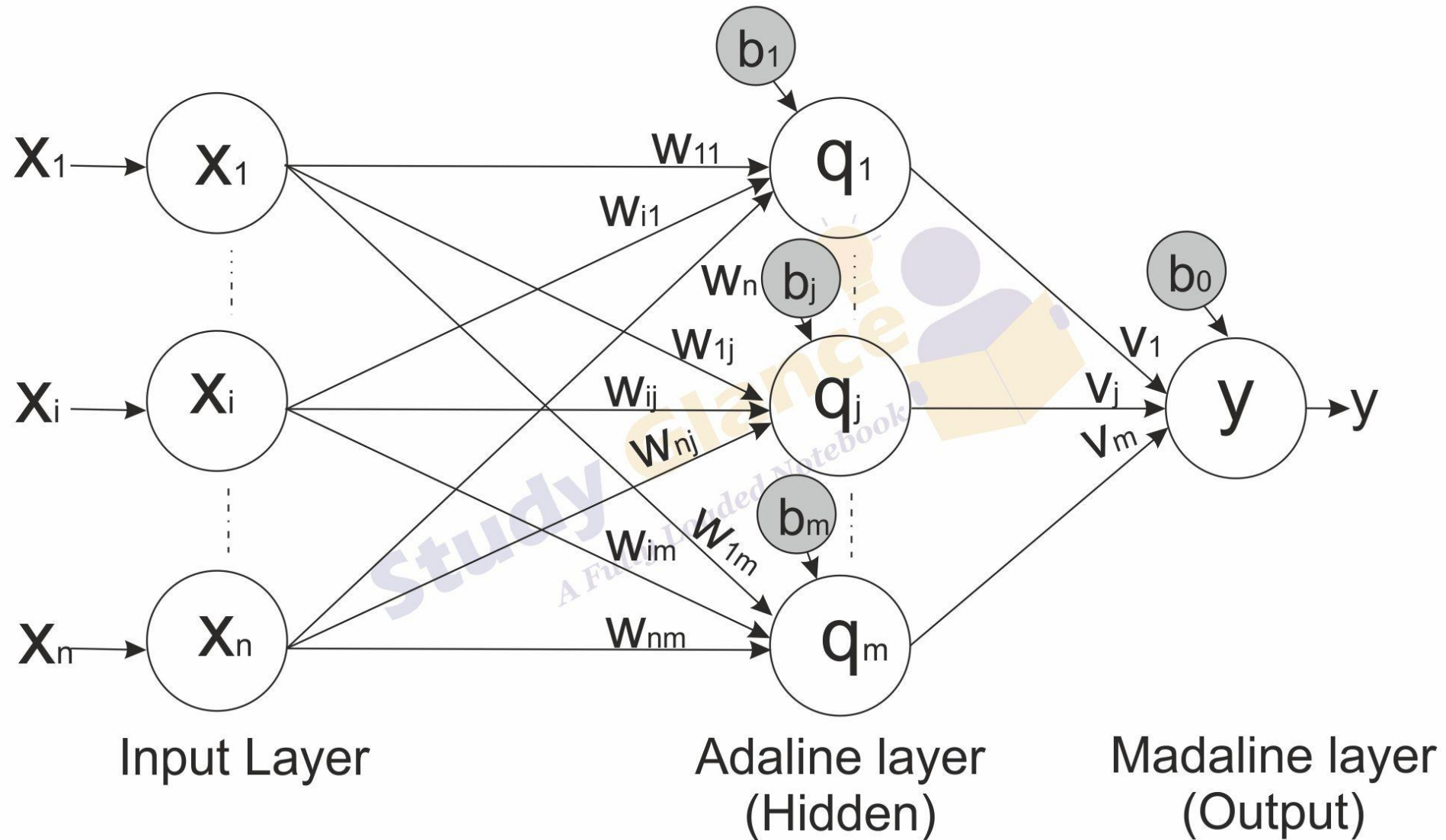
Multiple Adaptive Linear Neuron (Madaline)

- Madaline which stands for **Multiple Adaptive Linear Neuron**, is a network which consists of many Adalines in parallel. It will have a single output unit. Some important points about Madaline are as follows –
- It is just like a multilayer perceptron, where Adaline will act as a hidden unit **between the input and the Madaline layer**.
- The weights and the bias between the input and Adaline layers, as in we see in the Adaline architecture, are adjustable.
- The Adaline and Madaline layers have fixed **weights and bias of 1**.
- Training can be done with the help of **Delta rule**.

Multiple Adaptive Linear Neuron (Madaline)

- It consists of “n” units of input layer and “m” units of Adaline layer and “1” unit of the Madaline layer.
- Each neuron in the Adaline and Madaline layers has a bias of excitation “1”.
- The Adaline layer is present between the input layer and the Madaline layer; the Adaline layer is considered as the hidden layer.

Multiple Adaptive Linear Neuron (Madaline)



Multiple Adaptive Linear Neuron (Madaline) Training Algorithm

By now we know that only the weights and bias between the input and the Adaline layer are to be adjusted, and the weights and bias between the Adaline and the Madaline layer are fixed.

Step 0 – initialize the weights and the bias(for easy calculation they can be set to zero). also initialize the learning rate $\alpha(0, \alpha, 1)$ for simplicity α is set to 1.

Step 1 – perform steps 2-6 when stopping condition is false.

Step 2 – perform steps 3-5 for each bipolar training pair $s:t$

Step 3 – Activate each input unit as follows –

$$x_i = s_i(i = 1 \text{ to } n)$$

Step 4 – Obtain the net input at each hidden layer, i.e. the Adaline layer with the following relation –

$$Q_{inj} = b_j + \sum_i^n x_i w_{ij}(j = 1 \text{ to } m)$$

Here 'b' is bias and 'n' is the total number of input neurons.

Step 5 – Apply the following activation function to obtain the final output at the Adaline and the Madaline layer –

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Output at the hidden (Adaline) unit

$$Q_j = f(Q_{inj})$$

Final output of the network

$$\text{i.e. } y_{inj} = b_0 + \sum_{j=1}^m Q_j v_j$$

$$y = f(y_{in})$$

Step 6 – Calculate the error and adjust the weights as follows –

If $t \neq y$ and $t = +1$, update weights on Z_j , where net input is closest to 0 (zero)

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha(1 - Q_{inj})x_i$$

$$b_j(\text{new}) = b_j(\text{old}) + \alpha(1 - Q_{inj})$$

else If $t \neq y$ and $t = -1$, update weights on Z_k , whose net input is positive

$$w_{ik}(\text{new}) = w_{ik}(\text{old}) + \alpha(-1 - Q_{ink})x_i$$

$$b_k(\text{new}) = b_k(\text{old}) + \alpha(-1 - Q_{ink})$$

else if $y = t$ then

no weight updation is required.

Step 7 – Test for the stopping condition, which will happen when there is no change in weight or the highest weight change occurred during training is smaller than the specified tolerance.

Limitations of Perceptron

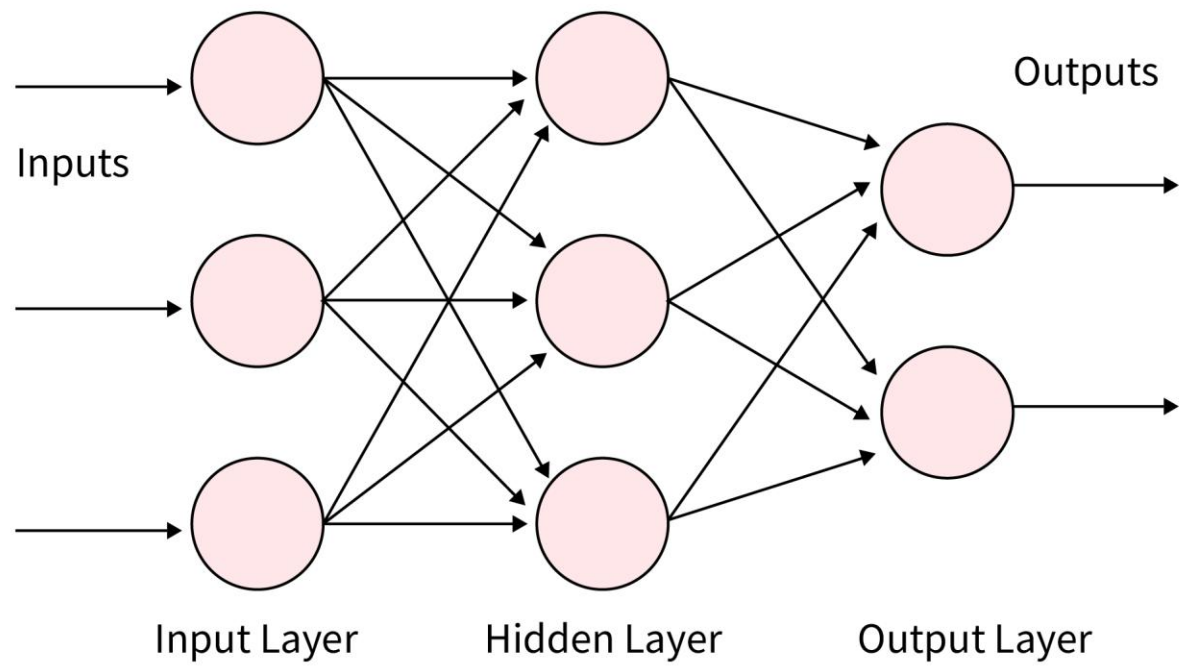
- The Perceptron was a significant breakthrough in the development of neural networks, proving that simple networks could learn to classify patterns. However, the Perceptron model has certain limitations that can make it unsuitable for some tasks:
- **Limited to linearly separable problems**
- **Struggles with convergence when handling non-separable data**
- **Requires labeled data for training**
- **Sensitive to input scaling**
- **Lacks hidden layers for complex decision-making**

UNIT IV

- **Multi-Layer Perceptron Networks:**
- Introduction, MLP with 2 hidden layers, Simple layer of a MLP, Delta learning rule of the output layer, Multilayer feed forward neural network with continuous perceptions, Generalized delta learning rule, Back propagation algorithm.

What is a Multilayer Perceptron?

- A **Multilayer Perceptron (MLP)** is a type of artificial neural network designed based on the biological neural networks found in the human brain.
- It consists of three main layers: an input layer, one or more hidden layers, and an output layer.
- Each layer comprises interconnected nodes, commonly referred to as neurons or perceptrons.
- The network is "multilayer" due to the presence of multiple hidden layers, distinguishing it from a simpler single-layer perceptron.



Structure of MLP

- **Input Layer**

Accepts raw data (features). Each neuron corresponds to one input variable.

- **Hidden Layers**

One or more layers of neurons where computations happen. Each neuron applies a weighted sum of inputs plus a bias, followed by a nonlinear activation function.

- **Output Layer**

Produces final predictions. The type of activation depends on the task:

- *Sigmoid / Softmax* → classification
- *Linear* → regression

Operation

- **Forward Propagation**

- Input is multiplied by weights and added with bias.
- Activation functions introduce non-linearity.
- Signals propagate layer by layer until the output.

- Formula for hidden neuron j :

- $$a_j = f\left(\sum_i w_{ij} x_i + b_j\right)$$

- **Backpropagation**

- Error is computed at the output.
- Gradient descent updates weights and biases using the chain rule.

Key Characteristics

- **Nonlinear modeling** → Can approximate any continuous function (Universal Approximation Theorem).
- **Supervised learning** → Trained with labeled data.
- **Activation functions** → Sigmoid, Tanh, ReLU, etc.

- **Advantages**

- Can solve complex nonlinear problems.
- Flexible for classification, regression, feature extraction.
- Basis for deep learning architectures.

- **Limitations**

- Computationally expensive with many hidden layers.
- Risk of **overfitting** on small datasets.
- Training requires large data and proper hyperparameter tuning.
- Less interpretable compared to simpler models.

Real-World Applications

- Handwritten digit recognition (MNIST).
- Medical diagnosis (disease classification).
- Speech and image recognition.
- Financial forecasting.

Gradient descent

- **Gradient descent** is the backbone of the learning process for various algorithms, including linear regression, logistic regression, support vector machines, and neural networks which serves as a fundamental optimization technique to minimize the cost function of a model by **iteratively adjusting the model parameters to reduce the difference between predicted and actual values, improving the model's performance.**

Generalized delta learning rule

- The **Generalized Delta Learning Rule (GDLR)** is the foundation of how error backpropagation works in training multilayer neural networks.
- It extends the **Delta Rule** (originally developed for single-layer perceptrons) to multi-layer feedforward networks that use differentiable activation functions.
- The **Generalized Delta Learning Rule** = Gradient Descent + Error Backpropagation.
- It generalizes the delta rule to handle **multi-layer, differentiable neural networks**.

Core Idea

- The rule adjusts the weights of a neuron to minimize the **error** between the actual output and the desired target output.
- It uses **gradient descent** to move in the direction that reduces the network's total error.

Weight Update Rule

- For a weight w_{ij} connecting neuron j to neuron i :
- $\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$
- Where:
- η : Learning rate
- $\frac{\partial E}{\partial w_{ij}}$: Gradient of the error with respect to the weight

Backpropagation Using GDLR

- **Forward pass:** Compute outputs layer by layer.
- **Backward pass:** Calculate error gradients starting from the output layer and propagate them backward.
- **Weight adjustment:** Update weights using the generalized delta rule.

For Output Layer Neurons

- $\Delta w_{ij} = \eta \cdot \delta_i \cdot o_j$
- Where:
- $\delta_i = (t_i - o_i)f'(net_i)$
- $f'(net_i)$:Derivative of the activation function at neuron i
- o_j :Output of the previous neuron

For Hidden Layer Neurons

- $\delta_h = f'(net_h) \sum_k \delta_k w_{hk}$
- Error signal (δ) of a hidden neuron depends on the weighted sum of error signals from the neurons in the next layer.

Advantages

- Works for **multi-layer networks** (unlike simple perceptron delta rule).
- Allows training with **non-linear activation functions**.
- Forms the backbone of the **Backpropagation algorithm**.

GENERALIZED DELTA LEARNING RULE

FORWARD PASS

compute outputs
layer by layer



CALCULATE ERROR

$$E = \frac{1}{2} \sum (t_k - o_k)^2$$



BACKWARD PASS

propagate error
backward



UPDATE WEIGHTS

$$\Delta w_{ij} = \eta \cdot \delta_i \cdot o_j$$

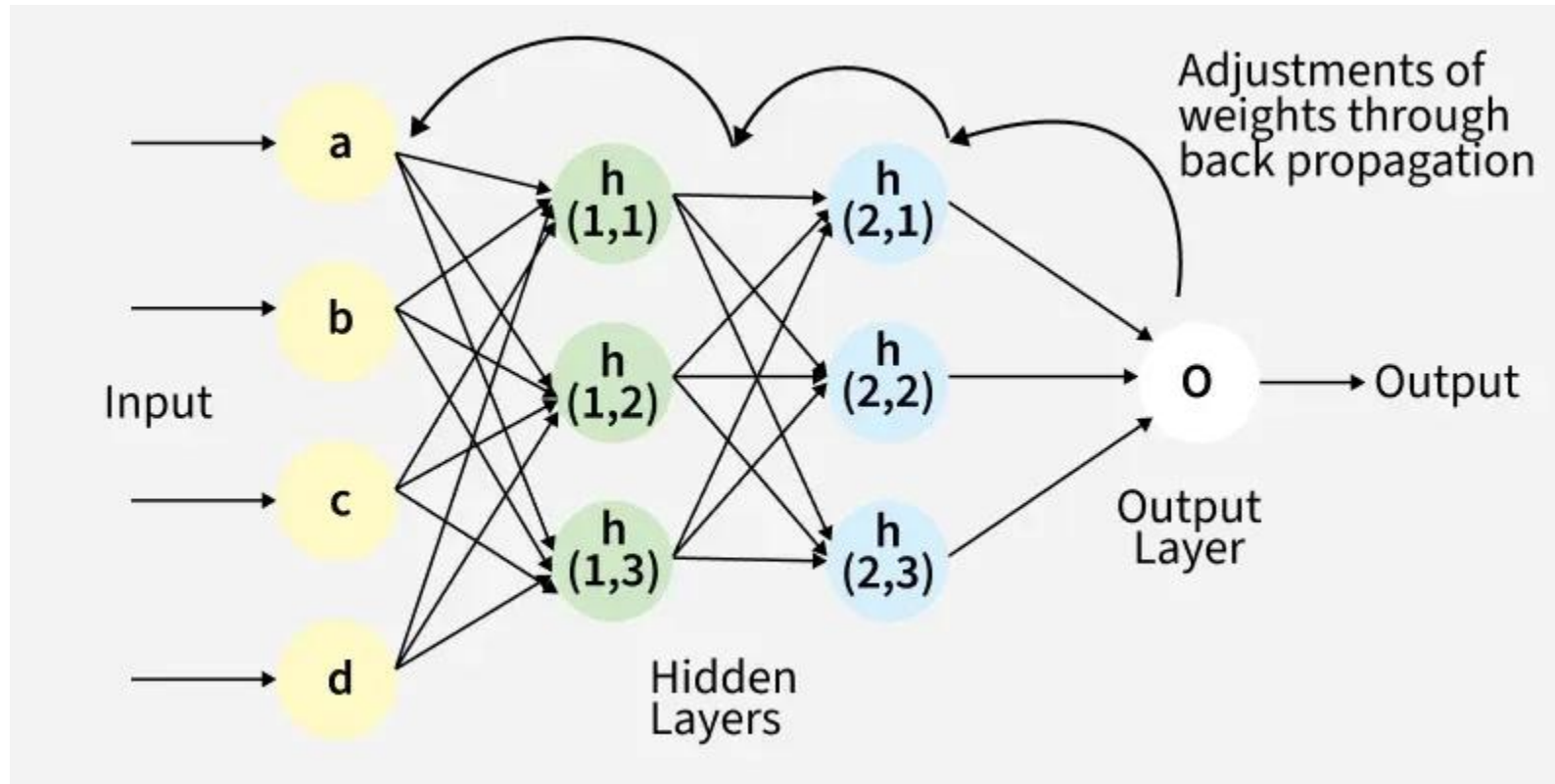
Backpropagation in Neural Network

- Back Propagation is also known as "Backward Propagation of Errors" is a method used to train neural network .
- Its goal is to reduce the difference between the model's predicted output and the actual output by adjusting the weights and biases in the network.

Backpropagation in Neural Network

- It works iteratively to adjust weights and bias to minimize the cost function.
- In each epoch the model adapts these parameters by reducing loss by following the error gradient.
- It often uses optimization algorithms like **gradient descent** or **stochastic gradient descent**.
- The algorithm computes the gradient using the chain rule from calculus allowing it to effectively navigate complex layers in the neural network to minimize the cost function.

Backpropagation in Neural Network



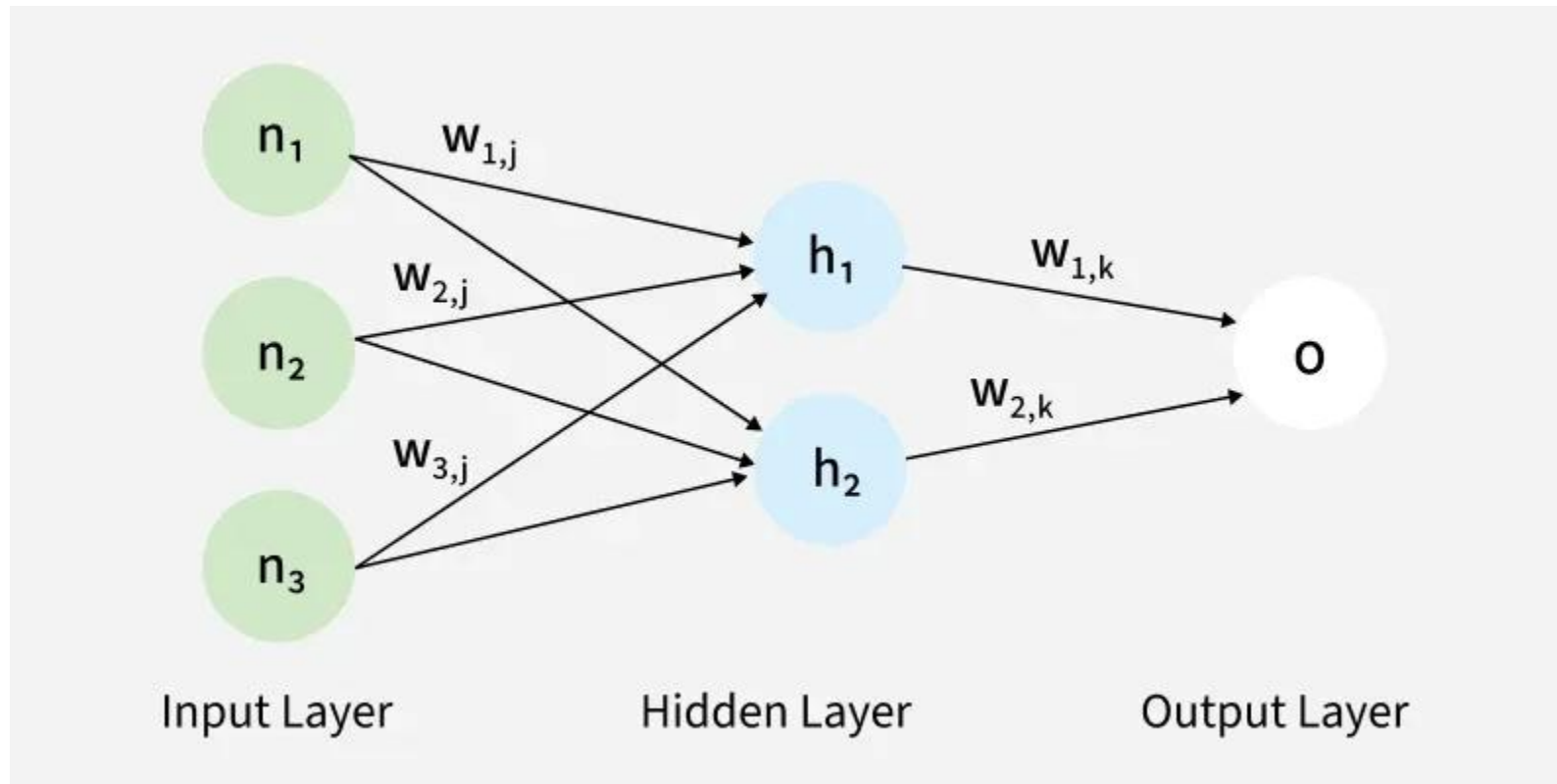
Back Propagation plays a critical role in how neural networks improve over time. Here's why:

- **Efficient Weight Update:** It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
- **Scalability:** The Back Propagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
- **Automated Learning:** With Back Propagation the learning process becomes automated and the model can adjust itself to optimize its performance.

Working of Back Propagation Algorithm

- The Back Propagation algorithm involves two main steps: the **Forward Pass** and the **Backward Pass**.
- **1. Forward Pass Work**
- In **forward pass** the input data is fed into the input layer. These inputs combined with their respective weights are passed to hidden layers. For example in a network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2. Before applying an activation function, a bias is added to the weighted inputs.
- Each hidden layer computes the weighted sum ('a') of the inputs then applies an activation function like **ReLU (Rectified Linear Unit)** to obtain the output ('o'). The output is passed to the next layer where an activation function such as **softmax** converts the weighted outputs into probabilities for classification.

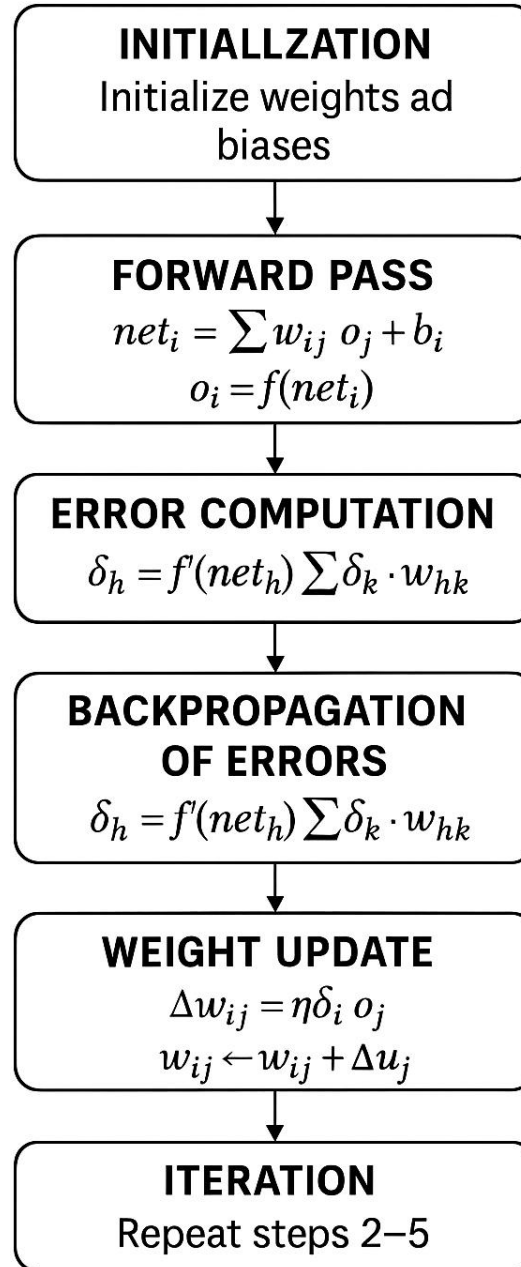
Forward Pass Work



2. Backward Pass

- In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the Mean Squared Error (MSE) given by:
- $$\text{MSE} = (\text{Predicted Output} - \text{Actual Output})^2$$
- Once the error is calculated the network adjusts weights using **gradients** which are computed with the chain rule.
- These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration.
- The backward pass continues layer by layer ensuring that the network learns and improves its performance.
- The activation function through its derivative plays a crucial role in computing these gradients during Back Propagation.

BACKPROPAGATION ALGORITHM



Advantages of Backpropagation

- **Ease of implementation:** accessible through multiple deep learning libraries such as Pytorch and Keras, facilitating its use in diverse applications.
- **Programming simplicity:** simplified coding with framework abstraction, reducing the need for complex mathematics.
- **Flexibility:** adaptable to diverse architectures, suitable for a broad spectrum of AI challenges.

Limitations and Challenges

- **Data quality:** poor data quality, including noise, incompleteness, or bias, can lead to inaccurate models, as backpropagation learns exactly what it is given.
- **Training duration:** backpropagation often requires extensive training time, which can be impractical when dealing with large networks.
- **Matrix-based complexity:** the matrix operations in backpropagation scale with the network size, which increases the computational demand, and potentially outstripping the available resources.

Design and implement the Backpropagation algorithm for a Multi-Layer Perceptron (MLP) consisting of three layers, where the input layer has four neurons, the hidden layer contains two neurons, and the output layer has a single neuron. Demonstrate step-by-step how the network updates its weights and biases during training using a learning rate of 0.8. Provide detailed justification for each computational step in the forward pass, error calculation, backward pass, and parameter update process.

1) network & notation

- layers: input (4) $\rightarrow$ hidden (2) $\rightarrow$ output (1)
- activations: sigmoid $\sigma(u)=1/(1+e^{-u})$ for hidden and output
- loss (per sample): $E = \frac{1}{2}(t - y)^2$

We'll **assume** small initial weights/biases (you can pick any small values):

- $W^{(1)}$ (hidden $\leftarrow$ input), shape 2×4

$$W^{(1)} = \begin{bmatrix} 0.1 & -0.2 & 0.3 & 0.4 \\ -0.4 & 0.2 & -0.1 & 0.1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

- $W^{(2)}$ (output $\leftarrow$ hidden), shape 1×2

$$W^{(2)} = [0.2 \quad -0.3], \quad b^{(2)} = 0.1$$

Training **sample** (for illustration):

$\mathbf{x} = [1, 0, 1, 0]$, target $t = 1$

2) forward pass (compute outputs)

Hidden pre-activations

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 0.1(1) + (-0.2)(0) + 0.3(1) + 0.4(0) \\ -0.4(1) + 0.2(0) + (-0.1)(1) + 0.1(0) \end{bmatrix} = \begin{bmatrix} 0.4 \\ -0.5 \end{bmatrix}$$

Hidden activations

$$\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)}) = \begin{bmatrix} \sigma(0.4) \\ \sigma(-0.5) \end{bmatrix} = \begin{bmatrix} 0.598688 \\ 0.377541 \end{bmatrix}$$

Output pre-activation

$$z^{(2)} = W^{(2)}\mathbf{a}^{(1)} + b^{(2)} = 0.2(0.598688) + (-0.3)(0.377541) + 0.1 = 0.106475$$

Output

$$y = \sigma(z^{(2)}) = \sigma(0.106475) = 0.526594$$

Loss

$$E = \frac{1}{2}(t - y)^2 = \frac{1}{2}(1 - 0.526594)^2 = 0.112057$$

3) backward pass (compute gradients)

We use squared error with sigmoid, so:

$$\delta^{(2)} = (y - t) y(1 - y) = (-0.473406)(0.526594)(0.473406) = -0.118017$$

Gradients for output layer

$$\frac{\partial E}{\partial W^{(2)}} = \delta^{(2)} (\mathbf{a}^{(1)})^\top = [-0.118017 \cdot 0.598688 \quad -0.118017 \cdot 0.377541] = [-0.070655 \quad -0.044556]$$

$$\frac{\partial E}{\partial b^{(2)}} = \delta^{(2)} = -0.118017$$

Hidden layer error terms

$$\boldsymbol{\delta}^{(1)} = \left((W^{(2)})^\top \delta^{(2)} \right) \odot \mathbf{a}^{(1)} \odot (1 - \mathbf{a}^{(1)})$$

With $W^{(2)} = [0.2, -0.3]$:

$$\delta_1^{(1)} = 0.2(-0.118017) \cdot 0.598688(1 - 0.598688) = -0.005671$$

$$\delta_2^{(1)} = (-0.3)(-0.118017) \cdot 0.377541(1 - 0.377541) = +0.008320$$

Gradients for hidden layer

$$\frac{\partial E}{\partial W^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top = \begin{bmatrix} -0.005671 \\ 0.008320 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} -0.005671 & 0 & -0.005671 & 0 \\ 0.008320 & 0 & 0.008320 & 0 \end{bmatrix}$$

$$\frac{\partial E}{\partial \mathbf{b}^{(1)}} = \boldsymbol{\delta}^{(1)} = \begin{bmatrix} -0.005671 \\ 0.008320 \end{bmatrix}$$

4) gradient descent updates ($\eta = 0.8$)

Update rule: $\theta \leftarrow \theta - \eta \frac{\partial E}{\partial \theta}$

Output layer

$$W_{\text{new}}^{(2)} = \begin{bmatrix} 0.2 & -0.3 \end{bmatrix} - 0.8 \begin{bmatrix} -0.070655 & -0.044556 \end{bmatrix} = \begin{bmatrix} 0.256524 & -0.264355 \end{bmatrix}$$

$$b_{\text{new}}^{(2)} = 0.1 - 0.8(-0.118017) = 0.194413$$

Hidden layer

$$W_{\text{new}}^{(1)} = \begin{bmatrix} 0.1 & -0.2 & 0.3 & 0.4 \\ -0.4 & 0.2 & -0.1 & 0.1 \end{bmatrix} - 0.8 \begin{bmatrix} -0.005671 & 0 & -0.005671 & 0 \\ 0.008320 & 0 & 0.008320 & 0 \end{bmatrix} = \begin{bmatrix} 0.104537 & -0.2 & 0.304537 & 0.4 \\ -0.406656 & 0.2 & -0.106656 & 0.1 \end{bmatrix}$$

$$\mathbf{b}_{\text{new}}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.8 \begin{bmatrix} -0.005671 \\ 0.008320 \end{bmatrix} = \begin{bmatrix} 0.004537 \\ -0.006656 \end{bmatrix}$$

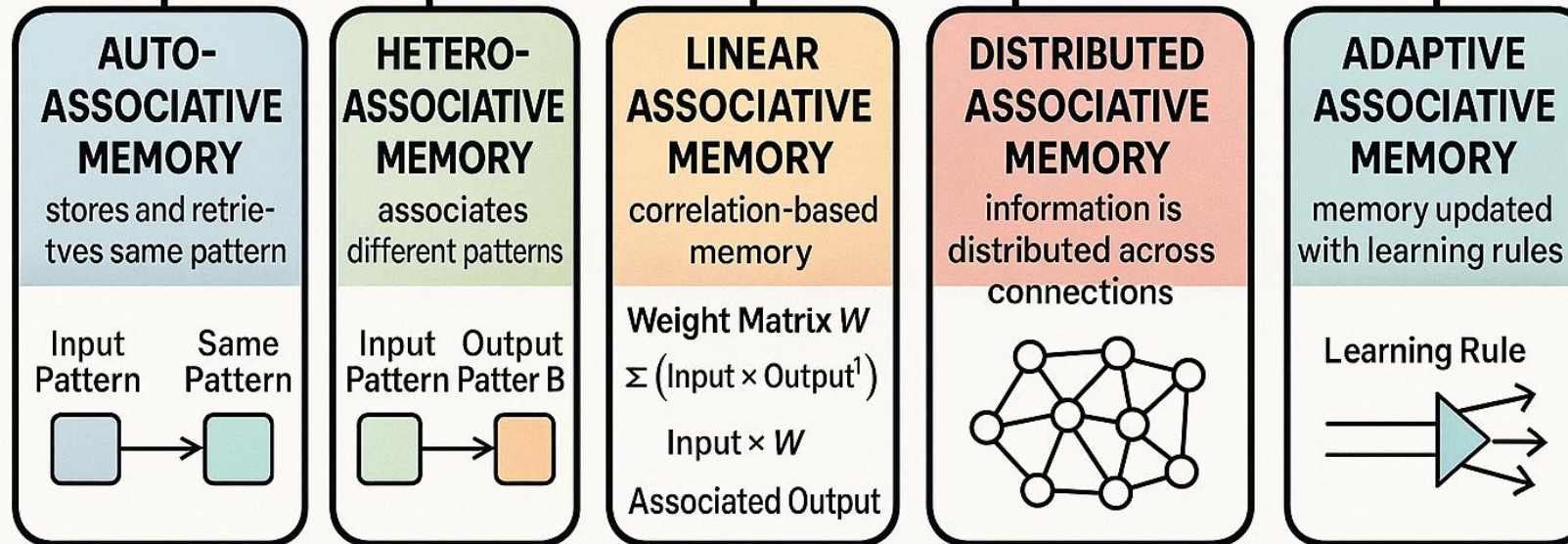
UNIT 5

- **Associative Memories:** Paradigms of Associative Memory, Pattern Mathematics, Hebbian Learning, General Concepts of Associative Memory, Bidirectional Associative Memory (BAM) Architecture, BAM Training Algorithms: Storage and Recall Algorithm, BAM Energy Function. **Architecture of Hopfield Network:** Discrete and Continuous versions, Storage and Recall Algorithm, Stability Analysis. Neural network applications: Process identification, control, fault diagnosis.

Paradigms of Associative Memory

- Associative memory (also called Content-Addressable Memory, CAM) refers to memory models and systems that retrieve stored information based on the content (pattern, cue, or partial input) rather than an explicit address.

Types of Pattern Associative in Neural Networks



1. Linear Associative Memory (LAM)

- Uses correlation-based learning between input and output patterns.
- The association is stored in the form of weight matrices (outer product rule).
- **Key idea:** A cue (input pattern) multiplied with the weight matrix retrieves the associated output.
- **Limitation:** Struggles with overlapping patterns; prone to cross-talk noise.

2. Autoassociative Memory

- Stores a pattern and retrieves it when given an incomplete or noisy version of the same pattern.
- Examples: **Hopfield Networks**, Hamming Networks.
- **Key idea:** Input and output are the same; memory corrects distorted inputs.
- **Applications:** Error correction, pattern completion.

3. Hetero associative Memory

- Associates one pattern with another (different input-output pairs).
- Examples: **Bidirectional Associative Memory (BAM)**.
- **Key idea:** Given one part (input or output), the system recalls the associated pair.
- **Applications:** Translation tasks, associative mapping.

4. Distributed Associative Memory

- Information is not localized but spread across a network of neurons/weights.
- Inspired by biological neural networks.
- **Key idea:** Robustness against noise and damage—partial failure does not erase memory completely.
- **Examples:** Hopfield nets, Boltzmann machines.

5. Content-Addressable Memory (Computational Paradigm)

- Hardware or algorithm-based memory systems that retrieve data by content, not address.
- Often implemented in hardware for high-speed lookups.
- **Applications:** Caches, routers, database search.

6. Adaptive Associative Memory

- Incorporates learning rules (Hebbian, Delta, Error-Correction).
- Allows dynamic updating of associations based on training.
- **Key idea:** Mimics human-like learning and adaptation.

- LAM**: Linear algebra-based association.
- Autoassociative**: Recall from noisy/incomplete input.
- Heteroassociative**: Map input to different output.
- Distributed**: Robust, fault-tolerant, biologically inspired.
- Content-addressable**: Computational, high-speed retrieval.
- Adaptive**: Learns and updates associations dynamically.

Summary Table

Type	Input-Output Relation	Example	Application
Auto-Associative	Input $\approx$ Output	Hopfield Net	Pattern completion
Hetero-Associative	Input $\neq$ Output	BAM	Translation, mapping
Linear Associative	Correlation-based	LAM	Simple associations
Distributed	Spread across network	Hopfield, Boltzmann	Robust recall
Adaptive	Learns dynamically	Hebbian/Delta	Adaptive learning

Bidirectional Associative Memory (BAM)

- Bidirectional Associative Memory (BAM) is a type of **recurrent neural network** that was introduced by Bart Kosko in 1988.
- It is designed to **store associations between pairs of patterns** (e.g., input–output pairs) and can recall one pattern when the other is presented.
- Unlike unidirectional associative memories, BAM allows information flow **in both directions**—from input to output and from output to input.

Bidirectional Associative Memory Architecture

- The architecture of BAM network consists of two layers of neurons which are connected by directed weighted, parallel interconnections.
- The network dynamics involve two layers of interaction.
- The BAM network iterates by sending the signals back and forth between the two layers until all the neurons reach equilibrium.
- The weights associated with the network are bidirectional.
- Thus, BAM can respond to the inputs in either layer.

Architecture of BAM

The BAM network consists of two layers:

1. Input Layer (X-Layer):

1. Represents the first pattern set.
2. Contains neurons $x_1, x_2, x_3, \dots, x_n$.
3. Each input neuron is connected to every output neuron.

2. Output Layer (Y-Layer):

1. Represents the second pattern set.
2. Contains neurons $y_1, y_2, y_3, \dots, y_m$.
3. Each output neuron is connected to every input neuron.

3. Bidirectional Connections:

1. The connections are **fully interconnected** but **not within the same layer** (no intra-layer connections).
2. Each weight w_{ij} connects an input neuron x_i to an output neuron y_j .
3. These weights are **symmetric**:

$$w_{ij} = w_{ji}$$

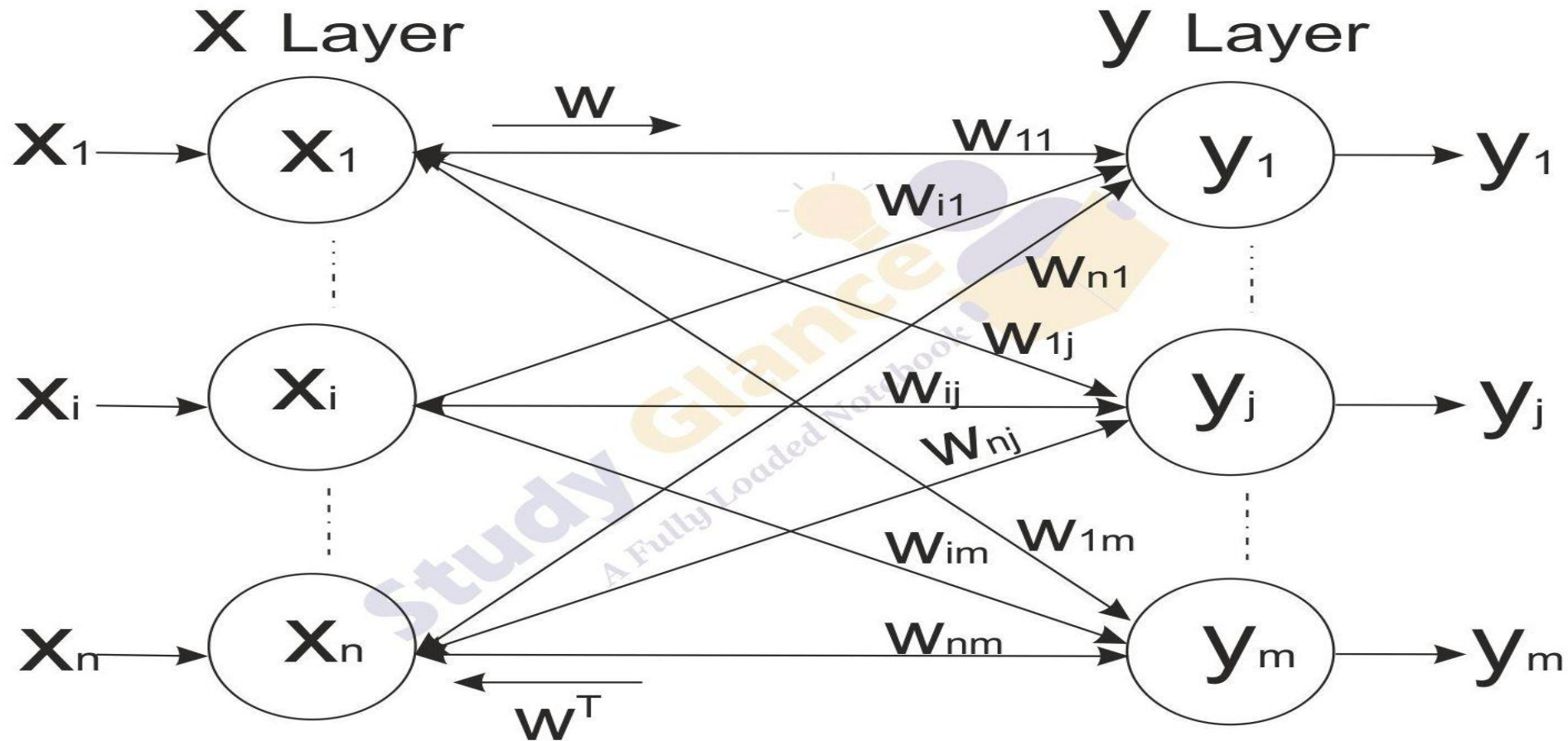


Figure shows a BAM network consisting of n units in X layer and m units in Y layer. The layers can be connected in both directions(bidirectional) with the result the weight matrix sent from the X layer to the Y layer is W and the weight matrix for signals sent from the Y layer to the X layer is W^T . Thus, the Weight matrix is calculated in both directions.

Working Mechanism

1. Learning Phase (Hebbian Learning Rule):

1. For each training pair (X, Y) , the weight matrix W is updated as:

$$W = \sum_{k=1}^p X_k^T Y_k$$

where p is the number of pattern pairs.

2. Recall Phase:

If an input vector X is presented:

$$Y = \text{sgn}(XW)$$

If an output vector Y is presented:

$$X = \text{sgn}(YW^T)$$

The process is repeated until the network reaches **stability** (no further changes in the output).

Example

Suppose we want to store the following associations:

$$\bullet X = [1, -, 1, 1] \leftrightarrow Y = [1, -1]$$

The weight matrix is computed as:

$$W = X^T Y = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} [1 \quad -1] = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$$

If input X is given, the network recalls Y , and vice versa.

Features

- **Bidirectional Recall:** Can retrieve input from output and output from input.
- **Associative Memory:** Stores mapping between two different sets of patterns.
- **Convergence:** The recall process continues until a stable state is reached.

Applications

- Pattern recognition
- Error correction in noisy inputs
- Memory retrieval in AI systems
- Associative data storage

Hopfield Neural Network

- A **Hopfield Neural Network (HNN)** is a type of **recurrent neural network** proposed by John Hopfield in 1982.
- It functions as an **auto-associative memory system** where the network can store patterns and retrieve them even from noisy or incomplete inputs.
- It is widely used in **pattern recognition, error correction, and associative memory tasks**.

Architecture of Hopfield Network

1. Single Layer of Neurons

1. The network consists of a single layer of neurons.
2. Each neuron acts as both an **input** and **output** unit.

2. Fully Connected Network

1. Every neuron is connected to every other neuron.
2. There are **no self-connections** (i.e., $w_{ii} = 0$)

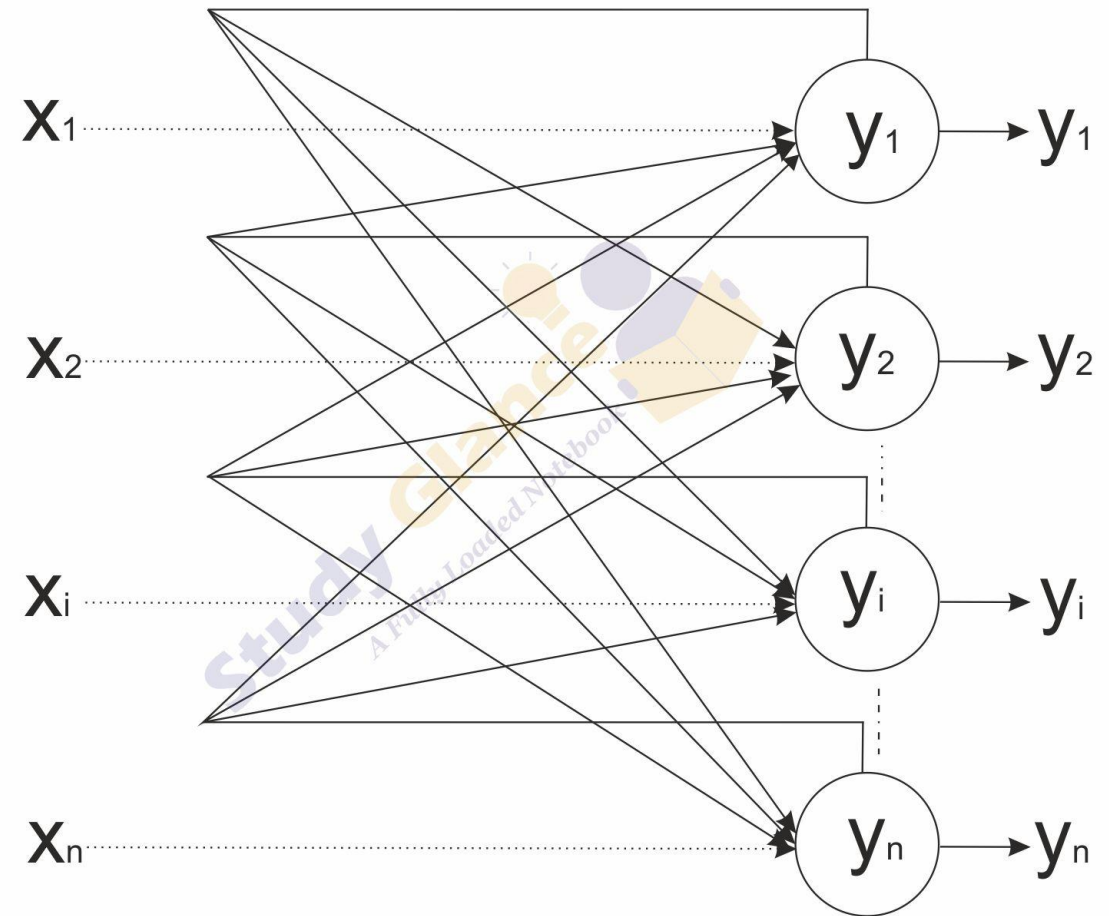
3. Symmetric Weights

1. The weight matrix is symmetric:

$$w_{ij} = w_{ji}$$

1. Binary States

1. Each neuron can take one of two possible values:
 1. Bipolar: +1 or -1
 2. Binary: 1 or 0



Working Principle

1. Learning Rule (Hebbian Rule)

1. The weight matrix is constructed using stored patterns:

$$w_{ij} = \frac{1}{N} \sum_{k=1}^p x_i^k x_j^k$$

1. where:

1. p = number of patterns,
2. N = number of neurons,
3. x_i^k = state of neuron i in pattern k .

1. Updating Neurons

1. Neurons update their state asynchronously or synchronously:

$$s_i(t+1) = \text{sgn} \left(\sum_{j=1}^N w_{ij} s_j(t) \right)$$

1. The network evolves until a **stable state (energy minimum)** is reached.

1. Energy Function

1. Hopfield networks minimize an energy function:
2. Ensures convergence to stable states.

Features

- **Content Addressable Memory (CAM):** Can recall full patterns from partial or noisy inputs.
- **Convergence:** Always converges to a stable state (local energy minima).
- **Symmetry Requirement:** Ensures stability.

Example

- If the network is trained with patterns like:
- $X^1 = [1, -, 11]$
- $X^2 = [-, 1, 1 - 1]$
- Then when a noisy version like $[1, 1, 1]$ is presented, the network recalls the closest stored pattern $[1, -, 11]$.

Applications

- Image and speech recognition
- Associative memory recall
- Error correction in communication
- Optimization problems

Types of Hopfield Network Architectures

- Discrete Hopfield Network
- Continuous Hopfield Network

1. Discrete Hopfield Network (DHN)

•Architecture

- Consists of binary neurons with states either +1 or -1 (sometimes 0 and 1).
- Fully connected recurrent network.
- Symmetric weight matrix with no self-connections ($w_{ii} = 0$).

•Working

- Updates neuron states asynchronously or synchronously.
- Uses **sign activation function**:

$$s_i(t + 1) = \text{sgn} \left(\sum_j w_{ij} s_j(t) \right)$$

- Converges to stable equilibrium points (local minima of energy).

•**Applications**: Associative memory, binary pattern recognition, error correction.

2. Continuous Hopfield Network (CHN)

•Architecture

- Extension of discrete model with continuous values for neuron states (not just binary).
- Uses **sigmoid activation function** for smoother dynamics.

•Working

- State of neuron i :

$$\tau_i \frac{du_i}{dt} = -u_i + \sum_j w_{ij} v_j + I_i$$

- where u_i is the internal potential, $v_i = f(u_i)$ is the output, and f is sigmoid.

•Applications: Solving **optimization problems** (like traveling salesman problem, scheduling).

Feature	Discrete Hopfield Network	Continuous Hopfield Network
Neuron States	Binary (+1, −1)	Continuous (0 to 1 or -1 to 1)
Activation Function	Sign Function	Sigmoid Function
Convergence	Finite steps	Continuous dynamics
Applications	Associative Memory, Pattern Recall	Optimization, Real-valued problems

- Design a bipolar auto-associative (Hopfield) memory to store $p_1 = [1, 1, 1, 1]^T$, and $p_2 = [-1, 1, 1, -1]^T$, using the outer-product learning rule. Derive the weight matrix with justified design choices (diagonal handling, normalization) and prove these patterns are fixed points under synchronous sign dynamics. Evaluate robustness to 1-bit noise, discuss spurious attractors via the energy function, and critique trade-offs between zeroing the diagonal vs. $1/N$ scaling.

1) Patterns (column vectors)

We want to store the two 4-D bipolar patterns

$$\boldsymbol{x}^{(1)} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad \boldsymbol{x}^{(2)} = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix}.$$

2) Learn with the outer-product rule

Unnormalized Hebbian weight matrix (before zeroing the diagonal):

$$\boldsymbol{W}_{\text{raw}} = \sum_{\mu=1}^2 \boldsymbol{x}^{(\mu)} \boldsymbol{x}^{(\mu)T} = \boldsymbol{x}^{(1)} \boldsymbol{x}^{(1)T} + \boldsymbol{x}^{(2)} \boldsymbol{x}^{(2)T}.$$

Compute each outer product:

- For $\boldsymbol{x}^{(1)}$:

$$\boldsymbol{x}^{(1)} \boldsymbol{x}^{(1)T} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

- For $x^{(2)}$ (with $[-1, 1, 1, -1]^T$):

$$x^{(2)}x^{(2)T} = \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}.$$

Sum them:

$$W_{\text{raw}} = \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}.$$

3) Remove self-connections (zero the diagonal)

Set $W_{ii} = 0$ for stability in recall:

$$W = \begin{bmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}.$$

This is the **final weight matrix**.

4) Test (synchronous update with sign activation)

Use $y = \text{sgn}(Wx)$, with $\text{sgn}(u_i) = \begin{cases} +1, & u_i \geq 0 \\ -1, & u_i < 0 \end{cases}$.

a) Test with $x = [1, 1, 1, 1]^T$:

$$h = Wx = \begin{bmatrix} 2 \\ 2 \\ 2 \\ 2 \end{bmatrix} \Rightarrow y = \text{sgn}(h) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \text{ (exactly recovered).}$$

b) Test with $x = [-1, 1, 1, -1]^T$:

$$h = Wx = \begin{bmatrix} -2 \\ 2 \\ 2 \\ -2 \end{bmatrix} \Rightarrow y = \text{sgn}(h) = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix} \text{ (exactly recovered).}$$

Result

- Weight matrix

$$W = \begin{bmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}$$